

MLP pour MNIST et moteur de différentiation automatique

Rapport de projet

Master CHPS – UVSQ 2025 - 2026

AUTEURS

Jianye Shi
jianye.shi@ens.uvsq.fr

Hao Qian
hao.qian@ens.uvsq.fr

Xiang Bian
xiang.bian@ens.uvsq.fr

Abdenmour Boulmis
abdenmour.boulmis@ens.uvsq.fr

ENCADREMENT

Aurélien Delval
Enseignant-chercheur

Table des matières

1	Introduction	1
1.1	Contexte du projet et objet d'étude	1
1.2	Base initiale et extensions de cette étape	1
1.3	Périmètre du rapport et organisation	1
2	Optimisation de l'opérateur de multiplication de matrices GEMM	2
2.1	Environnement expérimental	2
2.2	Budget de threads et protocole expérimental	2
2.3	Blocking et packing avec OpenMP	2
2.4	Architecture GotoBLAS	3
2.4.1	Modèle de blocking à trois niveaux	3
2.5	Réglage de GotoBLAS et stratégie parallèle multi-threads	4
2.5.1	Conception du micro-kernel	4
2.5.2	Résultats synthétiques	4
2.5.3	Configuration AVX2 finale et aperçu d'AVX-512	4
2.5.4	Stratégie parallèle multi-threads et réglage des paramètres pour GotoBLAS	5
2.5.5	Réglage des paramètres avec prise en compte des threads	5
2.5.6	Expérience de strong scaling	5
2.6	Principe d'implémentation Unrolled+Prefetch	6
3	Fonction de perte, optimiseurs et boucle d'entraînement	6
4	Implémentation et optimisation de l'opérateur de convolution	7
4.1	Implémentation initiale de la couche de convolution	7
4.2	Parallélisation de la couche de convolution et stratégie de découpage	8
4.3	Détails d'implémentation et choix d'ingénierie	8
4.4	Comparaison et métriques	8
5	Implémentation de l'architecture CNN	9
5.1	Structure et flux de données du CNN actuel	9
5.1.1	Vue d'ensemble	9
5.1.2	Flux de données, représentation mémoire et interface unifiée	10
5.2	Vérification à l'exécution	10
5.3	Extension vers une structure ResNet	10
5.3.1	Structure et stratégie de sous-échantillonnage	11
5.3.2	BasicBlock	11
6	Recherche automatique d'hyperparamètres avec Optuna	12
7	Portage vers Tiny-ImageNet et pipeline d'entrée	12
7.1	Motivation du portage et caractéristiques des données	12
7.2	Organisation du pipeline d'entrée	12
7.3	État actuel de l'entraînement et principales limites	13
7.4	Périmètre actuel et suite du travail	13
8	Entraînement distribué	14
8.1	Parallélisme de données synchrone et agrégation des gradients	14
8.2	Du paramètre par paramètre au gradient bucketing	14
8.3	Chevauchement entre communication et calcul	14
8.4	Compression des gradients : chemin d'implémentation et résultats intermédiaires	15

8.5	Expérience de montée en charge MPI	15
8.6	Résultats de profiling : décomposition des temps de communication	16
9	Chemin inspiré d'IST : attribution de paramètres et synchronisation périodique	16
9.1	Validation sur cluster et petite étude d'ablation	17
	Références	19
	Glossaire	20
A	Environnement expérimental GEMM, workloads et protocole de recherche	20
A.1	Environnement expérimental	20
A.1.1	Plateforme matérielle	20
A.1.2	Environnement logiciel	20
A.1.3	Protocole de mesure	20
A.2	Workloads utilisés pour le réglage mono-thread	21
A.3	Protocole de recherche en deux tours	21
B	Diagnostic des coûts d'ordonnancement et de synchronisation en configuration 16 threads	22
C	Le filtrage des paramètres OpenMP blocked / packing	23
D	Synthèse des résultats de réglage AVX2 en mono-thread	24
E	Dérivation de l'ensemble de candidats AVX2 pour les blocked sizes	25
E.1	Dérivation de m_r et n_r liée à l'implémentation	25
E.2	Modèle de cache et génération des candidats pour k_c	27
E.3	Génération des candidats de base mono-thread pour m_c/n_c	28
F	Complément sur la heatmap de réglage thread-aware $kCacheBlockM/N_c$	31
G	Expérience complémentaire de montée en charge sur matrices carrées fixes	32

1 Introduction

1.1 Contexte du projet et objet d'étude

La mise en oeuvre d'un système de réseaux de neurones ne se limite pas à l'architecture du modèle. Elle couvre aussi le runtime d'entraînement, les noyaux de calcul numérique et le pipeline d'entrée. Pour une tâche de classification d'images, développer depuis zéro un programme de réseau de neurones entraînable permet d'étudier, sur une base de code unique, les relations entre propagation avant, rétropropagation, mise à jour des paramètres et exécution expérimentale, tout en conservant un environnement maîtrisé pour les mesures de performance et les comparaisons d'implémentation.

Le présent rapport porte sur un système expérimental de réseaux de neurones implémenté en C++. Le système couvre actuellement deux familles de modèles, les perceptrons multicouches (MLP, multi-layer perceptron) et les réseaux convolutifs (CNN, convolutional neural network), ainsi qu'un runtime de différentiation automatique, des backends de multiplication de matrices et de convolution, une interface de chargement des données et un mécanisme de synchronisation pour les expériences d'entraînement distribué. L'analyse porte sur l'organisation de ces composants dans le périmètre actuel, leurs axes d'extension et leur usage expérimental, sans les présenter comme un framework générique de deep learning ni comme une plateforme d'entraînement stabilisée et exhaustive.

1.2 Base initiale et extensions de cette étape

Le travail préparatoire a déjà établi une base d'entraînement de perceptron multicouche sur MNIST [1], incluant la différentiation automatique en mode reverse, un graphe de calcul explicite, un noyau élémentaire de calcul matriciel et une boucle d'entraînement complète. Cet ensemble fournit une base fonctionnelle exécutable et sert de point de départ commun pour comparer plusieurs implémentations de calcul, étendre le chemin convolutif et observer les coûts du runtime.

Sur cette base, cette étape élargit le système, initialement limité à un prototype MLP/MNIST, en un support expérimental capable d'accueillir davantage de variantes techniques. Les extensions principales sont les suivantes : ajout d'un chemin CNN configurable pour les expériences sur modèles convolutifs ; comparaison et optimisation d'implémentations autour de la multiplication de matrices générale (GEMM, general matrix multiplication) et des opérateurs liés à la convolution ; réorganisation du runtime de différentiation automatique afin de distinguer plus clairement les coûts du framework et ceux des opérateurs ; intégration d'un chemin Tiny-ImageNet pour évaluer l'adaptation de l'implémentation actuelle à des entrées image plus lourdes et à une chaîne d'entraînement plus exigeante ; enfin, ajout d'un chemin expérimental de parallélisme de données synchrone fondé sur MPI (message passing interface) afin d'examiner l'organisation et les limites de la synchronisation distribuée des gradients dans l'implémentation actuelle.

1.3 Périmètre du rapport et organisation

La suite du rapport développe ces extensions dans l'ordre suivant. La première partie traite des implémentations GEMM et de leur méthode de comparaison, puis précise la configuration de calcul matriciel réutilisée dans les expériences suivantes. La deuxième présente le périmètre d'implémentation du chemin convolutif et de la structure CNN dans le code actuel. La troisième décrit l'intégration du jeu de données Tiny-ImageNet, le pipeline d'entrée et l'organisation du code de support à l'entraînement. La dernière aborde le chemin de synchronisation distribuée fondé sur MPI ainsi que la réorganisation du runtime de différentiation automatique. Les annexes complètent l'exposé par des expériences de diagnostic et des dérivations de paramètres utilisées

pour appuyer l'analyse et l'interprétation des résultats.

2 Optimisation de l'opérateur de multiplication de matrices GEMM

Dans le système actuel, GEMM (general matrix multiplication) est l'opérateur de calcul bas niveau central, utilisé à la fois dans les couches entièrement connectées du MLP et dans le schéma convolutif `im2col + matmul`. Son efficacité influe donc directement sur le coût global des expériences CNN, Tiny-ImageNet et d'entraînement distribué.

Ce chapitre présente, compare et optimise plusieurs chemins GEMM du projet : `blas` (OpenBLAS), `omp` (version OpenMP de base implémentée au semestre précédent), `omp_blocked`, `omp_blocked_packab`, `omp_gotoblas_avx2` et `omp_gotoblas_avx512`. L'objectif est d'adapter le workload GEMM à la machine cible à l'aide du parallélisme OpenMP, du blocking, du packing et d'une organisation de type GotoBLAS avec micro-kernels SIMD.

Les questions traitées sont les suivantes : quel gain apporte le parallélisme OpenMP de base, dans quelle mesure le blocking cache et le packing améliorent les accès mémoire, si l'organisation hiérarchique de type GotoBLAS est adaptée au workload du projet, et si les micro-kernels AVX2/AVX-512 restent stables selon la forme des matrices et le nombre de threads. Les résultats servent principalement à fixer la configuration `matmul` réutilisée dans les expériences ultérieures.

2.1 Environnement expérimental

Sauf mention contraire, les expériences de ce chapitre utilisent l'environnement matériel, la configuration logicielle, le protocole de mesure et les réglages d'affinité CPU décrits en annexe A. Les comparaisons GEMM utilisent par défaut des configurations de 1 à 8 threads, correspondant aux 8 cœurs physiques, avec `OMP_PROC_BIND=true` et `OMP_PLACES=cores`. Seules les expériences de contrôle sur le SMT/hyper-threading utilisent une configuration à 16 threads.

2.2 Budget de threads et protocole expérimental

Avant de comparer les implémentations GEMM, une série d'expériences de contrôle sur le SMT a servi à fixer le budget de threads. La conclusion est qu'une configuration à 16 threads (SMT) dégrade les performances à la fois avec OpenBLAS et avec l'implémentation OpenMP de base. La cause principale est l'amplification des coûts d'ordonnancement et d'attente de synchronisation sur des unités d'exécution physiques partagées par l'hyper-threading (voir annexe B).

En conséquence, toutes les expériences suivantes reprennent cette configuration de référence, afin d'écarter des comparaisons entre backends les fluctuations d'ordonnancement observées sous SMT. La comparaison de montée en charge avec contraintes d'affinité figure en annexe B.

2.3 Blocking et packing avec OpenMP

Par rapport à la référence `omp`, cette section évalue dans quelle mesure le **blocking** et le **packing** améliorent encore la localité de travail de chaque thread en parallèle.

Du point de vue de la localité temporelle, le blocking vise à réutiliser autant que possible les sous-blocs de A , B et C déjà présents en cache avant leur éviction. Sans cela, des données encore peu exploitées peuvent être remplacées par de nouveaux accès.

Le packing complète le blocking en recopiant les panneaux ou sous-blocs appelés à être relus dans un tampon contigu et réordonné. Il échange ainsi une copie supplémentaire contre des accès plus réguliers, moins de strides et une résidence en cache mieux contrôlée, ce qui réduit

les cache / TLB miss et améliore l'utilisation de la bande passante [2, 3].

Le réglage détaillé et la procédure de sélection pour `omp_blocked`, `omp_blocked_packb` et `omp_blocked_packab` sont regroupés en annexe C. Sur la plateforme étudiée, les résultats indiquent que la meilleure configuration de `omp_blocked` se rapproche d'un blocking résident en L2, tandis que les variantes avec packing présentent un jeu de données optimal plus proche d'une organisation orientée L1.

À partir de ce constat, la suite se concentre sur un chemin de type GotoBLAS plus structuré afin d'évaluer son adaptation à la machine et au workload considérés.

2.4 Architecture GotoBLAS

Le packing présenté plus haut améliore déjà la localité des données et prépare efficacement le calcul au niveau des registres. L'implémentation suivante ajoute, sur cette base, une couche de micro-kernel afin de mieux exploiter le SIMD au niveau registre, dans une organisation de type GotoBLAS/BLIS [2, 3].

2.4.1 Modèle de blocking à trois niveaux

GotoBLAS décompose $C = A \cdot B$ (de taille $M \times K \times N$) en trois niveaux de boucles imbriquées et un micro-kernel interne [2, 3], comme l'illustre la figure 1.

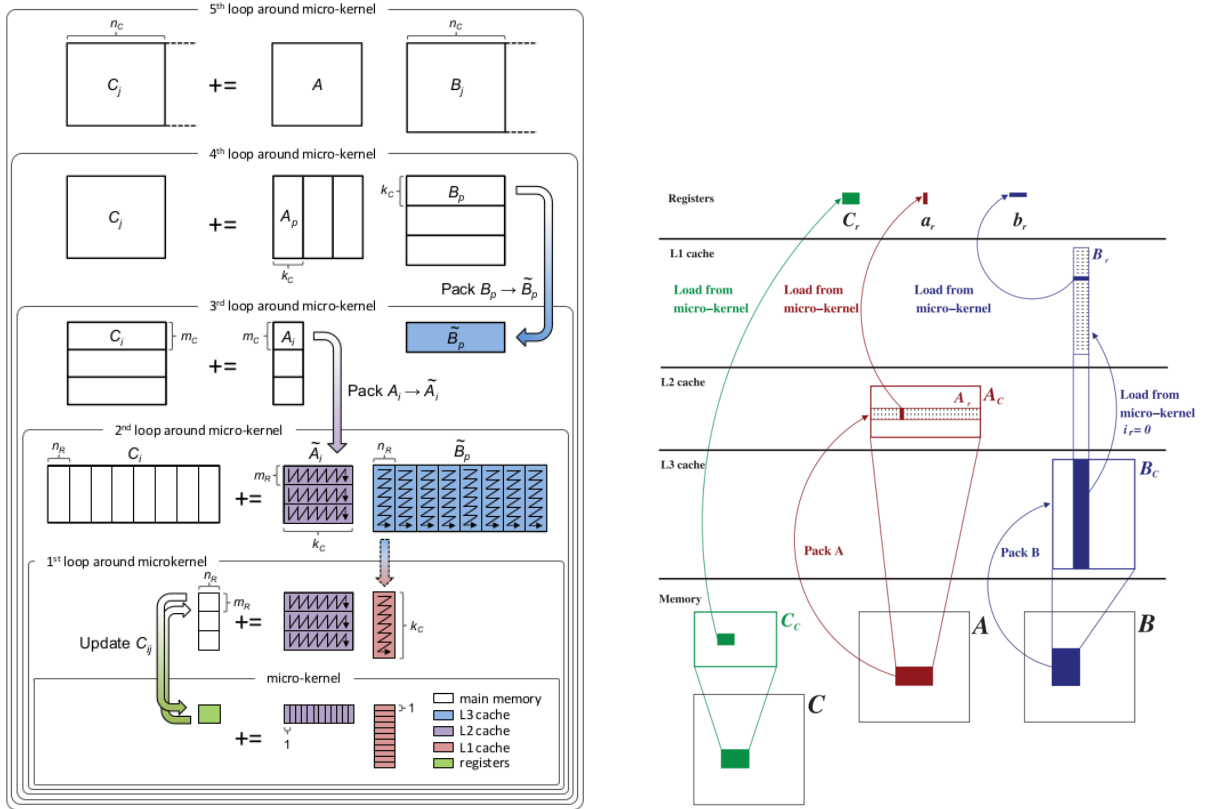


FIGURE 1 – À gauche : organisation hiérarchique de l'algorithme GEMM de type GotoBLAS/BLIS autour du micro-kernel, reproduite de [4, Fig. 1]. À droite : schéma des mouvements de données et du placement des panneaux dans la hiérarchie mémoire pour une implémentation BLIS/GotoBLAS, reproduit de [5, Fig. 4].

Les trois niveaux externes de blocking et de packing visent à compacter le jeu de données et à le maintenir dans le niveau de cache adapté, tandis que le micro-kernel interne maximise, au niveau des registres, la réutilisation de ces données packées [2, 3].

1. **Couche externe (dimension N_c)** : parcours de la dimension N avec un pas N_c pour charger dans le LLC (cache L3) les panneaux de B de taille $K \times N_c$;

2. **Couche intermédiaire (dimension K_c)** : parcours de la dimension K avec un pas K_c pour charger en cache L2 les panneaux packés de B de taille $K_c \times N_c$;
3. **Couche interne (dimension M_c)** : parcours de la dimension M avec un pas M_c pour charger en cache L1 les panneaux packés de A de taille $M_c \times K_c$;
4. **Micro-kernel (tile registre $M_r \times N_r$)** : opération la plus interne, avec accumulation d'un bloc de C de taille $M_r \times N_r$ dans les registres SIMD.

Ce modèle hiérarchique suppose que chaque niveau produit des tiles suffisamment réguliers. Lorsque M est faible ou que le nombre de threads est élevé, le nombre de lignes réellement attribuées à chaque thread après découpage selon M peut devenir inférieur à la hauteur du micro-kernel M_r . Les macro-blocs et blocs registres sont alors moins bien remplis, et le programme entre plus souvent dans les traitements de reste (*remainder loop*), ce qui réduit l'efficacité parallèle et la réutilisation cache attendues du blocking à trois niveaux.

Les paramètres de bloc cache sont estimés à partir de la capacité des panneaux : le panneau de A , de taille $M_c \times K_c \times 4$ octets, cible L1 (32 KiB par coeur), et celui de B , de taille $K_c \times N_c \times 4$ octets, cible L2 (1 MiB par coeur). Les valeurs par défaut sont actuellement $K_c = 384$, un nombre de lignes effectif $M_c = 64$ (facteur 8, à partir de $M_r = 8$) et $N_c = 448$. Ces paramètres sont ensuite affinés expérimentalement dans la section 2.5.

2.5 Réglage de GotoBLAS et stratégie parallèle multi-threads

La recherche de paramètres s'appuie sur les formes GEMM issues des réseaux de neurones du projet. Comme l'espace combinatoire de $(M_r, N_r, K_c, M_c, N_c)$ est trop large, la sélection est effectuée par étapes : réduction initiale des candidats, puis essais mono-thread sur les formes conservées.

2.5.1 Conception du micro-kernel

Le micro-kernel AVX2 en simple précision est organisé par blocs de 8 colonnes selon la dimension N , afin de correspondre à la largeur vectorielle YMM et de limiter les traitements de bord. Dans le chemin `float32`, un registre YMM de 256 bits couvre 8 valeurs `float`. L'implémentation privilégie donc des formes où N_r est multiple de 8.

Le détail du protocole de recherche, de la réduction de l'espace $(M_r, N_r, K_c, M_c, N_c)$ et des résultats par workload est reporté dans les annexes A, D et E. Le corps du texte ne retient ici que les conclusions utiles aux expériences multi-threads.

2.5.2 Résultats synthétiques

En mono-thread, les formes `4x16` et `8x8` sont les plus compétitives sur les workloads représentatifs. La première domine plusieurs formes de couches entièrement connectées, tandis que la seconde reste plus robuste sur une partie des workloads convolutionnels. Une figure représentative et le tableau récapitulatif des meilleures configurations par famille figurent en annexe (figure 11, tableau 8).

Par rapport à OpenBLAS, l'écart se situe typiquement entre 10 et 24% sur les formes courantes ($N \geq 64$), mais atteint 66 à 78% sur les très petits N , où les traitements de bord deviennent dominants. Pour disposer d'une configuration fixe unique dans les expériences multi-threads suivantes, le chemin AVX2 retient donc `avx2_8x8` comme compromis principal.

2.5.3 Configuration AVX2 finale et aperçu d'AVX-512

Au vu de ce compromis mono-thread, les expériences multi-threads suivantes utilisent la configuration AVX2 fixe suivante :

$$\text{avx2_8x8}, \quad K_c = 384, \quad M_c = 64, \quad N_c = 448$$

où $M_c = 64$ correspond au nombre effectif de lignes (constante de code `kCacheBlockM= 8`, valeur effective = `kCacheBlockM` $\times$ $M_r = 8 \times 8 = 64$). Cette configuration sert de point fixe pour les comparaisons multi-threads.

Le chemin AVX-512 suit le même schéma sur le chemin de données unifié `float32/Scalar=float`, avec un micro-kernel `avx512_8x32`. La configuration finale retenue est : `avx512_8x32`, $K_c = 160$, $M_c = 64$, $N_c = 448$ (M_c correspond à 64 lignes effectives ; $K_c = 160$ est une valeur issue du réglage expérimental, transmise à l'exécution via `MATMUL_KC=160`, contrairement à la constante compilée `kCacheBlockK= 384` du chemin AVX2). La dérivation détaillée des ensembles candidats pour le blocked-size AVX2 est donnée en annexe E.

2.5.4 Stratégie parallèle multi-threads et réglage des paramètres pour GotoBLAS

Le réglage mono-thread détermine la meilleure forme de noyau et les meilleurs paramètres de bloc. En multi-threads, l'implémentation conserve les valeurs M_c , N_c et K_c fournies à l'exécution comme paramètres de blocking cache, puis introduit à l'étage d'ordonnancement une granularité de tâche dépendante du nombre de threads : le code calcule un `task_m` plus fin sur la dimension ic/M afin de générer davantage de tâches sur les petites matrices et de réduire l'inactivité des threads [6].

2.5.5 Réglage des paramètres avec prise en compte des threads

Dans le code actuel, la prise en compte du nombre de threads agit surtout sur la granularité d'ordonnancement, et non sur les paramètres de bloc cache eux-mêmes : dans la région parallèle, le programme calcule un `task_m` plus fin selon le nombre de threads. Les expériences de *strong scaling* qui suivent utilisent donc uniquement la configuration finale à blocked-size fixe.

Les expériences de *strong scaling* utilisent la configuration fixe suivante : pour le chemin AVX2, `avx2_8x8`, $K_c = 384$, $M_c = 64$, $N_c = 448$; pour le chemin AVX-512, `avx512_8x32`, $K_c = 160$, $M_c = 64$, $N_c = 448$. La heatmap thread-aware `kCacheBlockM/N_c` donnée en annexe (figure 12) confirme que la zone favorable reste concentrée sur `kCacheBlockM= 8` (soit $M_c = 64$ effectif), tandis que la direction N_c forme un plateau relativement plat.

2.5.6 Expérience de strong scaling

L'expérience de *strong scaling* compare, à taille de problème fixe et pour $T \in \{1, 2, 4, 8\}$, GotoBLAS AVX2 / AVX-512 et OpenBLAS sur 9 workloads issus de MLP et de CNN.

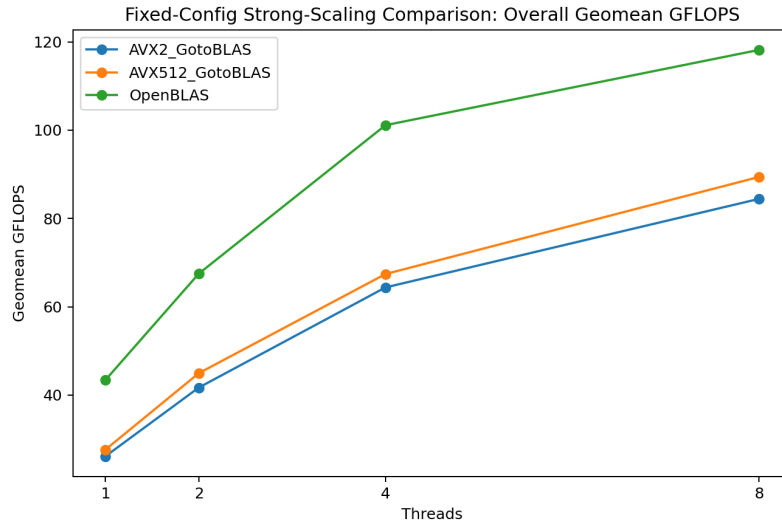


FIGURE 2 – Expérience de strong scaling : moyenne géométrique des GFLOPS pour chaque nombre de threads, calculée sur 9 workloads.

La figure 2 agrège ces 9 workloads par moyenne géométrique. OpenBLAS y conserve les GFLOPS les plus élevés, mais les chemins GotoBLAS AVX2 et AVX512 montent plus régulièrement avec le nombre de threads : à 8 threads, ils atteignent tous deux un speedup d'environ $3.23\times$, contre $2.72\times$ pour OpenBLAS. À l'échelle des workloads individuels, OpenBLAS reste le plus souvent devant ; les chemins GotoBLAS servent surtout à comparer le blocking, le packing et l'organisation OpenMP.

2.6 Principe d'implémentation Unrolled+Prefetch

Après le réglage de base du chemin AVX-512, le code a été étendu par une variante **unrolled+prefetch**. Cette version conserve la structure blocked+packed existante et ajoute seulement du déroulage de boucle (par exemple `GEMM_UNROLL_4`) et du prefetch explicite dans le micro-kernel AVX-512 et dans la phase d'accès aux panneaux packés correspondants. L'objectif est de réduire davantage l'attente mémoire dans la partie la plus interne de l'exécution. Cette variante agit sur le chemin d'accès séquentiel aux panneaux packés A/B , avec le même périmètre statistique que dans les sections précédentes.

Une expérience complémentaire sur matrices carrées fixes, utile surtout comme contrôle sur tailles synthétiques, est reportée en annexe G.

3 Fonction de perte, optimiseurs et boucle d'entraînement

Dans le point d'entrée principal de l'entraînement, le chemin de classification utilise `CrossEntropyLoss` : le modèle produit des logits de classe, puis la perte applique en interne un softmax numériquement stable et l'entropie croisée. Une implémentation de `MSELoss` reste présente dans le code, mais n'est pas utilisée ici.

Du côté des optimiseurs, l'implémentation actuelle propose trois variantes : `SGD`, `MomentumSGD` [7] et `AdamW` [8].

Pour `SGD`, les paramètres sont mis à jour directement à partir du gradient courant :

$$\theta_{t+1} = \theta_t - \eta g_t.$$

`MomentumSGD` ajoute un terme de vitesse, qui lisse exponentiellement l'historique des gradients afin de stabiliser la direction de mise à jour :

$$v_{t+1} = \mu v_t - \eta g_t, \quad \theta_{t+1} = \theta_t + v_{t+1},$$

La correction de Nesterov peut être activée selon la configuration. `AdamW` maintient en plus des estimations du premier et du second moment du gradient et découple le *weight decay* de la mise à jour adaptative.

Une fois le graphe de calcul construit, l'optimiseur est sélectionné à partir des paramètres de ligne de commande. À chaque itération, les gradients sont remis à zéro puis, après le backward et, le cas échéant, la synchronisation distribuée, mis à l'échelle selon la taille du batch global avant la mise à jour des paramètres. `Trainer::runEpoch(...)` regroupe les chemins d'entraînement et d'évaluation : le premier exécute propagation avant, calcul de la perte, rétropropagation et mise à jour ; le second se limite à la propagation avant et au calcul des métriques. Cette boucle sert ensuite de base commune aux expériences CNN, ResNet et distribuées.

La figure 3 regroupe les courbes d'entraînement sur 20 epochs pour ces trois optimiseurs appliqués au MLP sur MNIST.

La figure 3 montre que `SGD` converge plus lentement et reste en retrait en précision finale, tandis que `MomentumSGD` obtient ici la meilleure précision de test ; `AdamW` atteint un niveau proche,

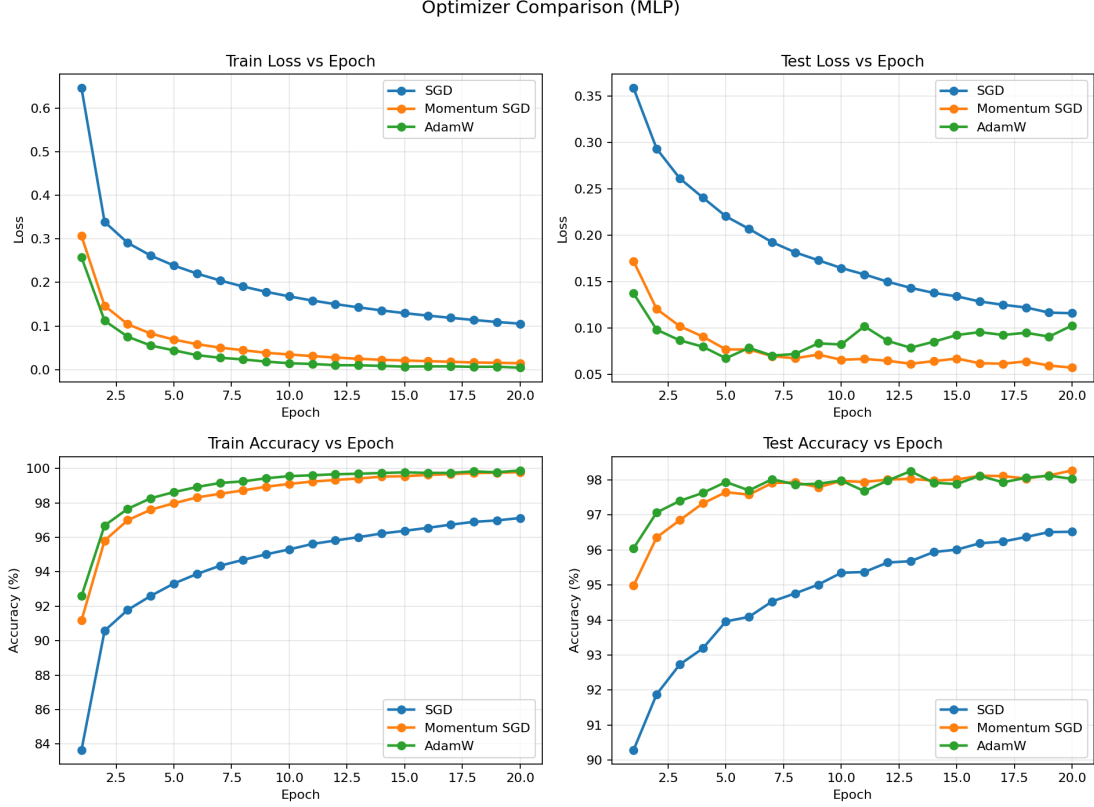


FIGURE 3 – Comparaison des pertes et des précisions d'entraînement/test pour trois optimiseurs sur le MLP MNIST (20 epochs). La précision finale en test est de 96.52% pour SGD, 98.27% pour MomentumSGD et 98.03% pour AdamW.

avec une convergence plus rapide au début.

4 Implémentation et optimisation de l'opérateur de convolution

Une convolution 2D applique, en chaque position, une somme pondérée sur un voisinage local, avec partage des poids sur toute l'image. Elle extrait ainsi des motifs locaux avec moins de paramètres qu'une couche entièrement connectée ; dans l'implémentation actuelle, elle suit un schéma `im2col/GEMM/col2im` [9] afin de réutiliser le backend GEMM disponible, en pratique OpenBLAS.

4.1 Implémentation initiale de la couche de convolution

Dans le backend de référence, `Conv2DLayer` matérialise ce choix en regroupant via `im2col` les voisinages locaux de l'entrée en lignes de matrice, puis en réécrivant la propagation avant d'une convolution comme une opération GEMM :

$$\begin{aligned}
 \text{im2col}(X) &\in \mathbb{R}^{(N \cdot H_{out} \cdot W_{out}) \times (C_{in} \cdot k_H \cdot k_W)}, \\
 W &\in \mathbb{R}^{C_{out} \times (C_{in} \cdot k_H \cdot k_W)}, \\
 Y &= \text{im2col}(X) \cdot W^T + b. \\
 Y &\in \mathbb{R}^{(N \cdot H_{out} \cdot W_{out}) \times C_{out}}, \quad \text{reshape}(Y) \in \mathbb{R}^{N \times C_{out} \times H_{out} \times W_{out}}.
 \end{aligned}$$

Ce choix réutilise le chemin `MathOps::matmul`. Dans l'implémentation actuelle, le coût total de la couche se répartit entre `im2col`, GEMM, la réorganisation de la sortie, le calcul des gradients `dW/dX` et `col2im`. Dans cette section, le fil conducteur reste le chemin avant principal `im2col/GEMM/bias+reshape`, tandis que `col2im` est traité dans la passe arrière (gradient `dX`) au sein du coût global de la couche.

4.2 Parallélisation de la couche de convolution et stratégie de découpage

Dans l'implémentation CNN actuelle du projet, la couche de convolution suit le chemin `im2col + GEMM`. La performance de la couche n'est donc pas déterminée par GEMM seul : le coût total inclut aussi `im2col`, `col2im` et les GEMM du backward. L'objectif n'est pas d'accélérer un kernel isolé, mais d'améliorer le débit de la chaîne complète en réduisant le surcoût de réagencement, en améliorant la montée en charge multi-threads et en limitant les pertes sur les petites tâches.

Le choix central de parallélisation concerne alors la dimension de découpage. En théorie, les tâches peuvent être réparties selon le batch, les canaux de sortie, la dimension spatiale de sortie ($H_{out} \times W_{out}$) ou la réduction, mais l'implémentation privilégie l'indépendance des éléments de sortie : chaque thread traite des blocs disjoints afin d'éviter verrous et opérations atomiques. Dans le chemin avant, `im2col` est découpé par échantillons et par blocs spatiaux, tandis que la phase GEMM réutilise directement la stratégie parallèle du backend sous-jacent ; dans le backward, `dW`, `dX` et `col2im` restent eux aussi organisés autour de blocs de sortie afin de limiter les écritures conflictuelles.

Un découpage plus fin n'est toutefois pas toujours meilleur. Lorsque la granularité devient trop faible, les coûts fixes de création de threads, de synchronisation et d'ordonnancement augmentent fortement, en particulier sur de petites entrées comme celles de MNIST. La stratégie retenue cherche donc un compromis entre parallélisme disponible et densité de calcul par tâche.

4.3 Détails d'implémentation et choix d'ingénierie

Sur le plan de l'implémentation, l'optimisation se concentre sur trois points. Le premier consiste à améliorer la continuité des accès mémoire : l'ordre des boucles et les calculs d'indices de `im2col/col2im` sont ajustés pour que les boucles internes parcourent autant que possible des adresses contiguës, ce qui augmente l'utilisation des lignes de cache et réduit le coût des calculs d'adresse. Le deuxième concerne la gestion des tampons temporaires : les buffers intermédiaires sont réutilisés lorsque c'est possible afin de limiter les allocations et désallocations répétées pendant l'entraînement, et donc de réduire les fluctuations de l'allocateur. Le troisième porte sur la stratégie d'ordonnancement des threads : un découpage statique plus stable est privilégié lorsque la charge est régulière, afin de réduire le coût d'ordonnancement du runtime et les migrations de threads.

Ces modifications ont un trait commun : elles réorganisent le chemin d'exécution sans modifier la sémantique mathématique. Les résultats des passes avant et arrière restent donc alignés avec l'implémentation de base, et les optimisations se limitent à l'organisation du calcul, à la disposition des données et à l'ordonnancement parallèle, ce qui simplifie la vérification de la correction et préserve la compatibilité avec le flux actuel d'autodiff et d'optimisation.

4.4 Comparaison et métriques

Les comparaisons reprennent le même protocole reproductible que dans les sections précédentes, avec formes d'entrée, batch, environnement matériel, options de compilation et politique d'affinité CPU fixés. Les shapes étudiés correspondent d'abord aux deux convolutions de l'implémentation CNN actuelle sur MNIST et à leurs passes backward associées ; le nombre de threads

est généralement pris dans $\{1, 2, 4, 8\}$, avec échauffement, répétitions, médiane des temps et écart-type.

Les mesures suivent séparément `im2col`, le GEMM convolutif, la réorganisation de la sortie, `col2im` et les étapes du backward, afin de distinguer le gain sur le chemin principal et son effet sur la couche complète.

5 Implémentation de l'architecture CNN

Ce chapitre décrit l'intégration du chemin CNN dans le framework d'entraînement existant. L'idée directrice est de réutiliser l'infrastructure d'entraînement et de calcul déjà en place, en ne modifiant que l'abstraction réseau nécessaire à l'introduction du CNN.

5.1 Structure et flux de données du CNN actuel

5.1.1 Vue d'ensemble

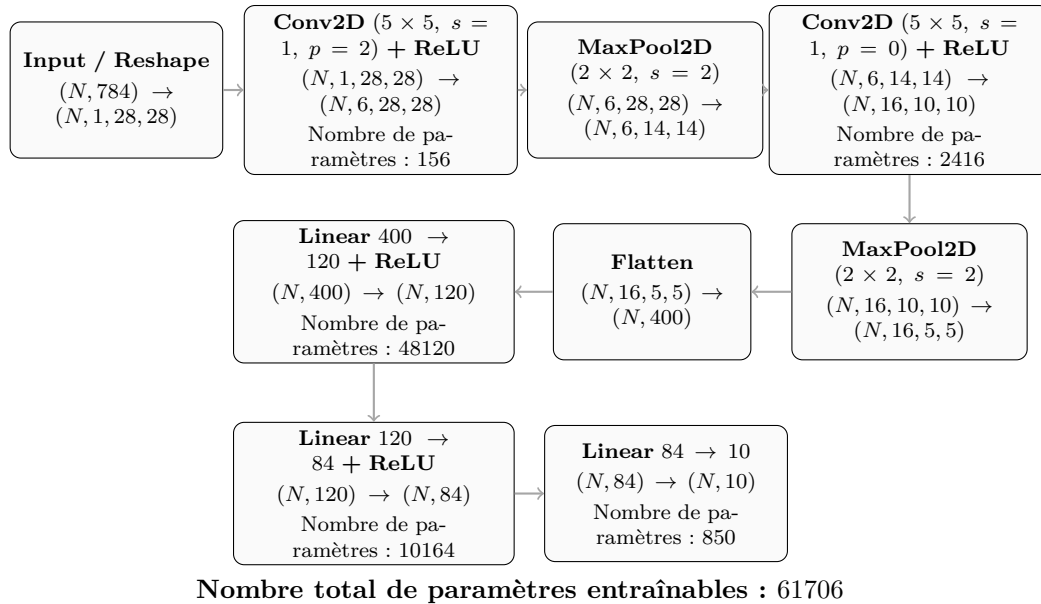


FIGURE 4 – Structure du chemin CNN actuel et évolution des shapes à chaque étape.

Les composants ajoutés ont chacun un rôle distinct. Les couches de convolution servent à extraire des motifs locaux par partage de poids ; avec des noyaux 5×5 , elles couvrent déjà une portion significative d'images MNIST 28×28 tout en gardant un nombre de paramètres modéré. Les couches de pooling assurent ensuite la réduction de résolution ; le choix du *max pooling* permet ici de conserver l'activation locale dominante sans introduire de paramètres supplémentaires, tout en diminuant rapidement le coût des couches suivantes. Enfin, la tête entièrement connectée convertit les caractéristiques compactées en logits de classe après mélange global des canaux et des positions spatiales restantes.

Le choix d'une architecture courte à deux convolutions, deux poolings et trois couches linéaires, inspirée de LeNet [1], vise ainsi à rester suffisamment représentatif pour MNIST, tout en gardant un coût d'entraînement et d'analyse modéré.

Le chemin CNN actuel fait d'abord passer chaque échantillon de $(N, 784)$ à une image monochrome de forme $(N, 1, 28, 28)$; ses transformations de forme sont résumées dans la figure 4.

L'ensemble du réseau compte environ 61706 paramètres entraînables. Cette échelle correspond au chemin CNN actuel pour MNIST et couvre quatre types de calcul : convolution, pooling,

mise à plat des caractéristiques et tête de classification.

5.1.2 Flux de données, représentation mémoire et interface unifiée

Le chemin avant de ce réseau suit la figure 4 : l'entrée est d'abord interprétée de $(N, 784)$ vers $(N, 1, 28, 28)$, puis traverse successivement deux convolutions, deux opérations de pooling, une mise à plat des caractéristiques et une tête de classification à trois couches entièrement connectées.

Le choix d'implémentation consiste à conserver **Matrix** comme stockage bas niveau unique, sans introduire de **Tensor4D** distinct. Le projet utilise uniformément des **float** en simple précision ; un tenseur logique à quatre dimensions (N, C, H, W) est aplati en mémoire dans une matrice bidimensionnelle **Matrix**($N, C \times H \times W$), puis réinterprété au moyen des paramètres de forme. Pour un élément (n, c, h, w) , l'indice linéaire s'écrit

$$\text{idx} = c \times H \times W + h \times W + w.$$

Ce choix évite de réécrire un nouveau système de stockage tensoriel et permet à la couche de convolution de réutiliser directement les interfaces matricielles existantes ; en contrepartie, chaque couche doit transmettre explicitement ses informations de forme et manipuler avec précaution les opérations de reshape et d'indexation. Après les deux convolutions et opérations de pooling, la carte de caractéristiques est mise à plat vers $(N, 400)$ avant d'être transmise à la tête de classification.

Pour éviter des boucles d'entraînement distinctes entre MLP, CNN et ResNet, ces modèles exposent tous l'interface unifiée **NeuralNetwork**, limitée à la propagation avant et à l'accès aux paramètres entraînaibles. Le **trainer**, la fonction de perte et l'optimiseur restent ainsi inchangés, tandis que **CNNNetwork** et **ResNetNetwork** n'ajoutent que la composition propre du réseau dans la hiérarchie « couches → réseau → entrée d'entraînement ».

5.2 Vérification à l'exécution

Modèle	Nombre d'épochs	Précision en test	Temps réel total
MLP	1	90.57%	0.96 s
MLP	5	94.28%	3.39 s
CNN	1	96.08%	25.54 s

TABLE 1 – Comparaison grossière du CNN et du MLP sur MNIST (**batch=256**, **seed=42**). La dernière colonne indique le temps réel total observé sur une exécution.

Le CNN atteint 96.08% après un epoch, contre 94.28% pour le MLP après cinq epochs. Son coût reste toutefois bien plus élevé : 25.54 s contre 0.96 s pour un epoch de MLP et 3.39 s pour cinq epochs.

5.3 Extension vers une structure ResNet

Au-delà du chemin CNN actuel, la capacité de cette architecture courte devient limitée dès que l'on vise des jeux de données plus grands et plus complexes. Le projet a donc intégré **ResNetNetwork**, afin d'introduire une structure plus profonde et plus facile à entraîner. Le principe central de ResNet [10] est la connexion résiduelle : un bloc sépare le calcul en une branche principale, qui applique les convolutions et produit un terme résiduel $F(x)$, et un chemin shortcut, qui transmet directement l'entrée. Dans le cas le plus simple, la sortie du bloc s'écrit

$$H(x) = x + F(x).$$

Autrement dit, le bloc n'apprend plus un mapping complet à partir de zéro, mais seulement la correction $F(x)$ à ajouter à l'identité. Lorsque les dimensions changent, le shortcut n'est plus exactement x mais une projection $P(x)$, et la sortie devient alors

$$y = F(x) + P(x).$$

Cette structure atténue les problèmes de dégradation du gradient dans l'entraînement des réseaux profonds et conserve une bonne aptitude à l'optimisation lorsque la profondeur augmente.

L'extension ResNet suit toujours l'interface unifiée `NeuralNetwork` et se branche donc directement sur la chaîne d'entraînement existante, sans boucle spécifique. Cette conception permet d'intégrer la structure résiduelle sans réécrire le framework d'entraînement, tout en vérifiant sa compatibilité avec le chemin principal d'autodiff.

5.3.1 Structure et stratégie de sous-échantillonnage

La configuration par défaut de `ResNetConfig` vise des entrées MNIST, avec une forme d'entrée $1 \times 28 \times 28$ et 10 classes. Le réseau commence par une couche de convolution de type stem, puis enchaîne trois stages ; leurs nombres de canaux sont respectivement 16, 32 et 64, et chaque stage contient deux blocs résiduels. Les dimensions spatiales évoluent selon $28 \times 28 \rightarrow 28 \times 28 \rightarrow 14 \times 14 \rightarrow 7 \times 7$, avant une couche entièrement connectée qui produit des logits de dimension 10.

5.3.2 BasicBlock

L'unité de base de l'implémentation ResNet actuelle est `BasicBlock`. Comme le résume la figure 5, une branche principale applique deux convolutions 3×3 ; son résultat est ensuite additionné à un shortcut avant l'activation finale σ .

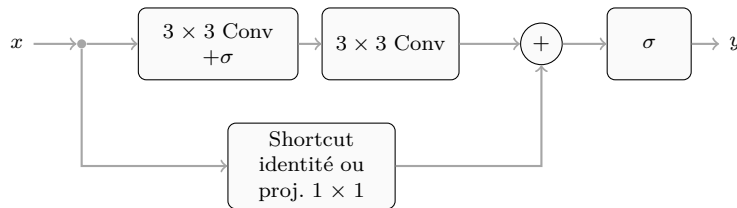


FIGURE 5 – Schéma d'un `BasicBlock` résiduel dans l'implémentation ResNet actuelle.

On note par $P(x)$ le chemin shortcut, qui vaut l'identité lorsque les dimensions sont inchangées, et une projection 1×1 sinon. Ici, σ désigne la fonction d'activation configurée dans le bloc, avec ReLU comme choix par défaut dans la configuration actuelle. Le calcul d'un bloc résiduel peut alors s'écrire :

$$\begin{aligned} F(x) &= W_2(\sigma(W_1x)), \\ y &= \sigma(F(x) + P(x)). \end{aligned}$$

Le passage d'un stage au suivant utilise une convolution de stride 2 pour le sous-échantillonnage, accompagnée d'une projection 1×1 sur le chemin shortcut afin de conserver la cohérence des dimensions spatiales et des canaux entre les deux branches. L'extension ResNet est déjà intégrée dans le code, avec une gestion explicite du shortcut identité et du shortcut projeté. En revanche, cette partie n'a pas encore fait l'objet d'un réglage systématique des hyperparamètres ni d'une campagne de benchmark approfondie. Pour viser de meilleures performances sur des jeux de données plus complexes, il restera à intégrer BatchNorm, un global average pooling et une configuration d'entraînement plus systématique.

6 Recherche automatique d'hyperparamètres avec Optuna

En plus du réglage manuel, le projet a introduit **Optuna** [11] pour organiser une recherche automatique d'hyperparamètres. Le point de départ est simple : les premières expériences sur MNIST ont déjà montré que le taux d'apprentissage, la taille de batch, la fonction d'activation et l'optimiseur influencent fortement la convergence. Avec l'ajout du chemin CNN, une exploration purement manuelle devient rapidement coûteuse. Le HPO est donc traité ici comme une extension directe du flux d'entraînement, et non comme un script auxiliaire ajouté après coup.

En pratique, chaque trial exécute un entraînement indépendant et prend `test_acc` comme objectif. La métrique d'évaluation peut être soit la meilleure précision de test observée pendant l'entraînement, soit la précision de test du dernier epoch :

$$\text{score} = \begin{cases} \max_e \text{test_acc}_e, & \text{metric=best,} \\ \text{test_acc}_E, & \text{metric=last.} \end{cases}$$

Le **study** associé est donc formulé en maximisation.

L'espace de recherche couvre les paramètres d'entraînement les plus sensibles : taux d'apprentissage, batch, activation, initialisation et optimiseur. La partie optimiseur suit une recherche conditionnelle afin d'éviter des combinaisons peu utiles et de concentrer le budget sur des régions plus pertinentes.

Les résultats sont sauvegardés avec leur configuration et leur contexte d'exécution. Le système prend aussi en charge le parallélisme intra-processus et un mode MPI worker. Parmi les résultats conservés, une campagne de 40 trials avec 3 epochs par trial atteint `best test_acc = 0.9891`. La configuration manuelle par défaut atteint 98.45% avec le même budget ; le gain du meilleur trial reste donc modéré, à +0.46 point.

7 Portage vers Tiny-ImageNet et pipeline d'entrée

Avec l'intégration du chemin CNN et de l'entraînement distribué, le champ expérimental s'étend de MNIST à **Tiny-ImageNet**, une variante réduite d'ImageNet [12]. Ce portage augmente non seulement l'échelle des entrées, mais introduit aussi une charge système plus complète, avec décodage JPEG, entraînement convolutif et synchronisation distribuée ; il expose donc plus directement le domaine de validité et les goulots d'étranglement de l'implémentation actuelle.

7.1 Motivation du portage et caractéristiques des données

MNIST suffit pour valider les chemins de base liés au MLP, à la fonction de perte et à la différentiation automatique, mais ne couvre pas des scénarios plus lourds comme le chargement d'images JPEG, le calcul CNN ou la synchronisation MPI. À l'inverse, **Tiny-ImageNet** fournit environ 100000 images d'entraînement et environ 10000 images de validation, ce qui convient mieux à l'évaluation d'un pipeline d'entrée plus complexe et d'un flux d'entraînement plus chargé. Les expériences actuelles utilisent une séparation entre jeu d'entraînement et jeu de validation, sans jeu de test indépendant intégré au chemin principal.

7.2 Organisation du pipeline d'entrée

Pour s'adapter aux entrées image de **Tiny-ImageNet**, le système abandonne le schéma unique utilisé sur MNIST, fondé sur un chargement intégral suivi d'un découpage par index. Une abstraction **BatchSource** est introduite entre le loader et la source de données bas niveau

afin de décrire, via une interface unifiée, la taille de l'échantillon, les dimensions d'entrée et le chargement d'un batch par index. Dans le cas de MNIST, cette abstraction peut se réduire à la copie de quelques lignes depuis une matrice déjà chargée; dans le cas de `Tiny-ImageNet`, elle devient un chargement et un décodage JPEG par batch pendant l'entraînement. Le loader réutilise les tampons de batch et conserve une logique unifiée pour les index et le mélange aléatoire.

7.3 État actuel de l'entraînement et principales limites

Le chemin `Tiny-ImageNet` est maintenant raccordé à la logique de chargement et à la boucle d'entraînement. Les évaluations mentionnées dans cette section correspondent à une configuration d'entraînement/évaluation avec séparation entre jeu d'entraînement et jeu de validation.

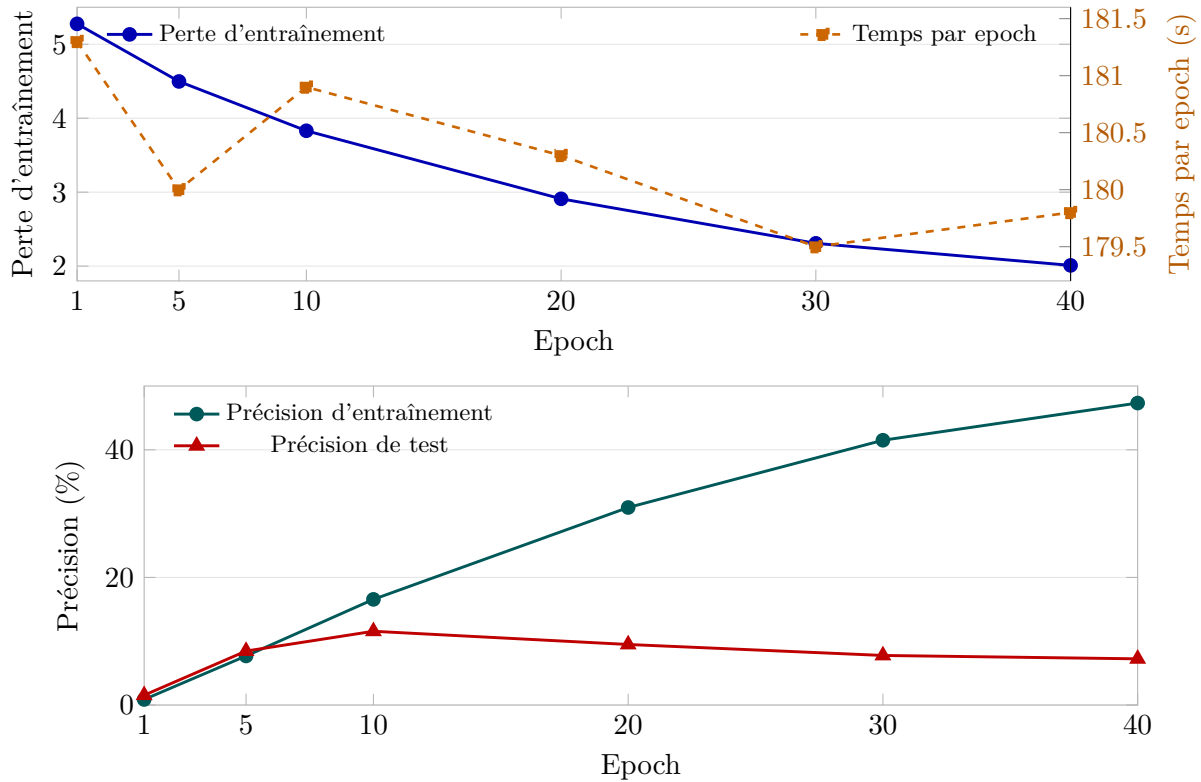


FIGURE 6 – Résumé des courbes d'entraînement sur 40 epochs pour `Tiny-ImageNet` (`world_size=4`, `batch=128`, synchronisation `per_param`). Le graphique du haut montre la perte d'entraînement et le temps par epoch; celui du bas présente la précision d'entraînement et la précision de test.

La figure 6 montre que la précision d'entraînement continue de monter jusqu'à 47%, alors que la précision de test commence à décroître dès le 5^e epoch et ne vaut plus que 7.24% au 40^e. Dans la configuration actuelle, avec une structure CNN de type LeNet [1] et sans augmentation de données, le modèle présente donc un surapprentissage marqué sur `Tiny-ImageNet`.

7.4 Périmètre actuel et suite du travail

À ce stade, ce chemin permet déjà de charger `Tiny-ImageNet` et d'exécuter l'entraînement de manière stable, mais des limites nettes subsistent encore. Le pipeline d'entrée ne couvre pas encore la prélecture asynchrone, le chevauchement avec le décodage, le cache ni une augmentation de données plus complète; côté modèle, le coût de `im2col/col2im`, du backward convolutif et des transferts entre convolution et GEMM reste à examiner. La suite du travail doit donc progresser en parallèle sur le système d'entrée et sur l'entraînement du modèle afin d'améliorer la précision de test sur `Tiny-ImageNet`.

8 Entraînement distribué

Cette section présente le chemin distribué actuel, du parallélisme de données synchrone aux réorganisations du runtime destinées à réduire le coût de synchronisation. L'enjeu est d'évaluer comment la granularité de communication, le chevauchement calcul/communication et la compression des gradients modifient le coût réel d'un step distribué.

8.1 Parallélisme de données synchrone et agrégation des gradients

L'entraînement distribué actuel repose sur un parallélisme de données synchrone : le modèle logique reste unique, mais il est répliqué sur chaque **rank** afin que plusieurs lots puissent être traités en parallèle, sur un même nœud ou sur plusieurs nœuds, chacun ne parcourant que sa partition locale de données [13]. Après chaque step, ces copies doivent être resynchronisées en agrégeant leurs gradients, de sorte que toutes appliquent ensuite la même mise à jour. Si l'on note g_r la somme des gradients obtenue par le rank r à un step donné, le gradient global est d'abord obtenu par réduction :

$$g_{\text{sum}} = \sum_{r=0}^{\text{world_size}-1} g_r.$$

La taille globale du batch s'écrit alors

$$B_{\text{global}} = \sum_{r=0}^{\text{world_size}-1} B_r.$$

Le gradient moyen global utilisé pour la mise à jour des paramètres est donc

$$g_{\text{global}} = \frac{1}{B_{\text{global}}} g_{\text{sum}} = \frac{1}{B_{\text{global}}} \sum_{r=0}^{\text{world_size}-1} g_r.$$

Tous les ranks appliquent ensuite ce même gradient moyen ; les copies du modèle restent donc cohérentes après chaque step. La difficulté n'est pas mathématique, mais organisationnelle : une réduction bloquante paramètre par paramètre multiplie les petites communications.

8.2 Du paramètre par paramètre au gradient bucketing

Pour réduire les coûts fixes de communication, le système introduit ensuite le gradient bucketing : plusieurs gradients de paramètres sont d'abord empaquetés dans un tampon contigu, puis la réduction est effectuée à l'échelle du bucket. La sémantique de l'entraînement ne change pas ; seule la granularité de synchronisation passe du paramètre individuel au groupe de paramètres, ce qui diminue le nombre d'appels de communication collective.

8.3 Chevauchement entre communication et calcul

Une fois le bucketing stabilisé, le système ajoute un chemin de chevauchement entre communication et calcul : dès que tous les gradients d'un bucket sont prêts, une réduction asynchrone peut être lancée avant la fin complète du backward. Le moment de synchronisation est ainsi avancé, depuis « exécution groupée après le backward » vers « exécution dès disponibilité des gradients », ce qui ouvre la possibilité de masquer une partie de l'attente liée à la communication.

8.4 Compression des gradients : chemin d’implémentation et résultats intermédiaires

L’objectif de la compression des gradients [14] est de réduire le coût de synchronisation tout en préservant la convergence. Le premier chemin, appliqué localement côté `train` après synchronisation, modifiait la représentation du gradient sans réduire le volume échangé par `AllReduce` ; il a donc été déplacé à la frontière de communication.

Dans cette itération, la logique de compression a été déplacée dans le runtime de synchronisation distribuée, selon le schéma

`compress` $\rightarrow$ `AllReduce` $\rightarrow$ `decompress` $\rightarrow$ `step`

comme cadre d’exécution, intégré de manière uniforme aux chemins `bucketed` et `overlap_bucketed`. L’implémentation associée ajoute des hooks d’encodage/décodage et des entrées de profiling décomposées pour distinguer les coûts d’encodage, de communication et de décodage ; le premier noyau de quantification, trop coûteux, a ensuite été remplacé par une implémentation plus légère fondée sur des opérations bit à bit. En parallèle, la stratégie de déclenchement de la compression a été corrigée : elle ne suit plus un intervalle entre buckets, mais un intervalle entre steps. Sur la plateforme actuelle, `interval=8` fournit un compromis satisfaisant entre coût ajouté et bénéfice observé.

Sous `world_size=2`, sur MNIST, avec MLP(128), `batch=64` et `seed=42`, les médianes obtenues sur trois répétitions sont les suivantes :

- **`bucketed`** : pour `fp16+i8` par rapport à la baseline, `avg_step_time_ms` vaut +1.84%, `samples_per_s` vaut -1.80%, et `test_acc` vaut -0.02% ;
- **`overlap_bucketed`** : pour `fp16+i8` par rapport à la baseline, `avg_step_time_ms` vaut +1.58%, `samples_per_s` vaut -1.54%, et `test_acc` vaut -0.02%.

Au vu de ces résultats, la configuration retenue à ce stade est : `--grad_compress 1 --grad_compress_mode fp16 --grad_compress_interval 8`.

Le cadre de compression à la frontière de communication est maintenant en place, mais le payload reste encore stocké en `float`. Les résultats valident donc surtout l’organisation du chemin ; le gain réel d’une communication effectivement compressée reste à mesurer.

8.5 Expérience de montée en charge MPI

Le tableau 2 présente les résultats obtenus sur le CNN MNIST avec `local_batch=64` et `seed=42` lorsque le nombre de processus varie. Dans cette configuration, `local_batch` est fixé pour chaque rank ; `global_batch` augmente donc avec `np`.

Nb processus	local batch	global batch	Précision en test	Temps (s)	Speedup relatif
1	64	64	93.79%	31.66	1.00×
2	64	128	91.93%	19.49	1.62×
4	64	256	88.63%	11.98	2.64×

TABLE 2 – Résultats de montée en charge multi-processus pour le CNN MNIST (1 epoch, `local_batch=64`, `seed=42`)

Lorsque le nombre de processus passe de 1 à 4, le temps d’exécution diminue de 31.66s à 11.98s, soit un speedup de 2.64×. La variation de précision observée dans ce tableau dépend à la fois de l’augmentation de `global_batch` et de la réduction du nombre de mises à jour par epoch.

Pour mieux comprendre ce qui limite le gain de montée en charge, la suite examine les résultats de profiling décomposés.

8.6 Résultats de profiling : décomposition des temps de communication

Les résultats de profiling présentés ici servent principalement à montrer comment se décomposent les postes `fwd_bwd`, `sync_total` et `sync_wait`, et non à établir une hiérarchie strictement comparable entre les différents modes.

Mode de synchronisation	fwd_bwd (s)	sync_total (s)	sync_wait (s)
bucketed	7.57	1.60	1.60
overlap_bucketed	8.25	1.09	0.97

TABLE 3 – Extrait illustratif des postes de temps dans le profile de synchronisation

`overlap_bucketed` reste, à ce stade, dans un périmètre de validation de correction. Les résultats fournis ici servent donc surtout à localiser les coûts de communication et d’attente ; le gain réel lié au chevauchement devra être vérifié séparément dans une configuration strictement alignée.

9 Chemin inspiré d’IST : attribution de paramètres et synchronisation périodique

Le chemin présenté ici ne reproduit pas l’IST standard (*Independent Subnet Training*), mais une variante inspirée de cette idée [15]. Chaque **rank** conserve une copie complète du modèle et exécute localement la propagation avant et la rétropropagation complètes ; la différence apparaît au moment de la mise à jour des paramètres. Le vecteur de paramètres complet est réparti entre les **ranks** au moyen d’un *ownership mask*, de sorte que chaque **rank** ne mette à jour, à l’intérieur d’une fenêtre de synchronisation, que les coordonnées dont il a la responsabilité. En fin de fenêtre, les incréments de paramètres sont fusionnés pour reconstruire un modèle global cohérent.

En notant $\theta \in \mathbb{R}^d$ le vecteur de paramètres aplati et R le nombre de processus MPI, on définit pour chaque **rank** r un masque binaire

$$m^{(r)} \in \{0, 1\}^d, \quad r = 0, \dots, R-1,$$

avec la contrainte qu’une coordonnée de paramètre appartient à un seul **rank** :

$$\sum_{r=0}^{R-1} m_i^{(r)} = 1, \quad i = 1, \dots, d.$$

Au début de la fenêtre de synchronisation s , tous les **ranks** partagent le même instantané θ_s . La longueur de fenêtre est

$$K = \text{ist_local_steps}.$$

Au pas local t , le **rank** r calcule sur son mini-batch le gradient du modèle complet

$$g_{s,t}^{(r)} = \nabla_{\theta} \ell(\theta_{s,t}^{(r)}; \mathcal{B}_{s,t}^{(r)}),$$

mais la mise à jour ne conserve que les coordonnées qui lui appartiennent :

$$\theta_{s,t+1}^{(r)} = \theta_{s,t}^{(r)} - \eta \left(m^{(r)} \odot g_{s,t}^{(r)} \right).$$

Le calcul avant/arrière reste donc complet ; le masque n’intervient qu’à la mise à jour des para-

mètres.

À la fin de la fenêtre, le **rank** r ne soumet que l'incrément associé à ses propres coordonnées :

$$\Delta_s^{(r)} = m^{(r)} \odot \left(\theta_{s,K}^{(r)} - \theta_s \right).$$

Comme les masques ne se chevauchent pas, l'incrément global s'obtient par un `MPI_Allreduce` en mode somme :

$$\Delta_s = \text{MPI_Allreduce}_{\text{sum}} \left(\Delta_s^{(r)} \right) = \sum_{r=0}^{R-1} \Delta_s^{(r)}.$$

Tous les **ranks** reviennent alors à la même valeur

$$\theta_{s+1} = \theta_s + \Delta_s.$$

Ce schéma n'est pas équivalent à un SGD synchrone classique. Dans un SGD synchrone ordinaire, chaque pas agrège les gradients calculés par les **ranks** sur le même vecteur logique de paramètres. Ici, à l'intérieur d'une fenêtre, chaque **rank** ne modifie que son bloc de paramètres, tandis que les autres coordonnées restent à leur valeur de début de fenêtre. Il faut donc l'interpréter comme une stratégie approchée visant à réduire la fréquence de synchronisation, et non comme une reformulation strictement équivalente du SGD synchrone pas à pas.

Dans l'implémentation actuelle, l'*ownership plan* est généré à partir de la forme des tenseurs de paramètres : les poids matriciels de type couche pleinement connectée sont découpés suivant la dimension de sortie, tandis que les paramètres unidimensionnels ou les autres tenseurs sont répartis élément par élément. Le paramètre `--ist_resample_every` permet de régénérer périodiquement cet *ownership plan* afin d'éviter une attribution fixe lorsque les partitions de données ne sont pas IID. Cette précaution améliore le comportement pratique, sans constituer une garantie d'équivalence avec le SGD synchrone.

Enfin, la communication reste aujourd'hui formulée comme un vecteur *dense* d'incréments passé à `MPI_Allreduce`. Même si de nombreuses positions valent zéro, le volume transmis par synchronisation reste donc proche de la dimension complète des paramètres. Le bénéfice principal observé à ce stade vient de la réduction de la fréquence de synchronisation, et non d'une baisse garantie du volume de communication par appel. Pour réduire aussi le coût d'une synchronisation individuelle, il faudrait introduire une communication creuse, par exemple en ne transmettant que les couples indice-valeur des coordonnées possédées.

9.1 Validation sur cluster et petite étude d'ablation

Sur le cluster `zen.univ-lille.fr`, nous avons d'abord vérifié que ce chemin inspiré d'IST était exécutable de bout en bout, puis mené une petite étude d'ablation. La configuration retenue pour cette validation est `Tiny-ImageNet` avec `ResNet`, `world_size=2`, `batch=64`, `lr=0.01` et l'optimiseur `momentum_sgd`.

La figure 7 présente une première ablation sur la longueur de fenêtre K avec `seed=42` et un seul epoch. Elle compare le chemin dense, IST ($K = 1$) et IST ($K = 4$) à la fois en précision de validation et en temps moyen par epoch. L'objectif de cette expérience n'est pas d'établir une conclusion définitive sur la précision, mais de confirmer que le chemin IST fonctionne de manière stable sur cluster et d'observer l'effet d'une fenêtre de synchronisation plus longue sur le coût d'exécution.

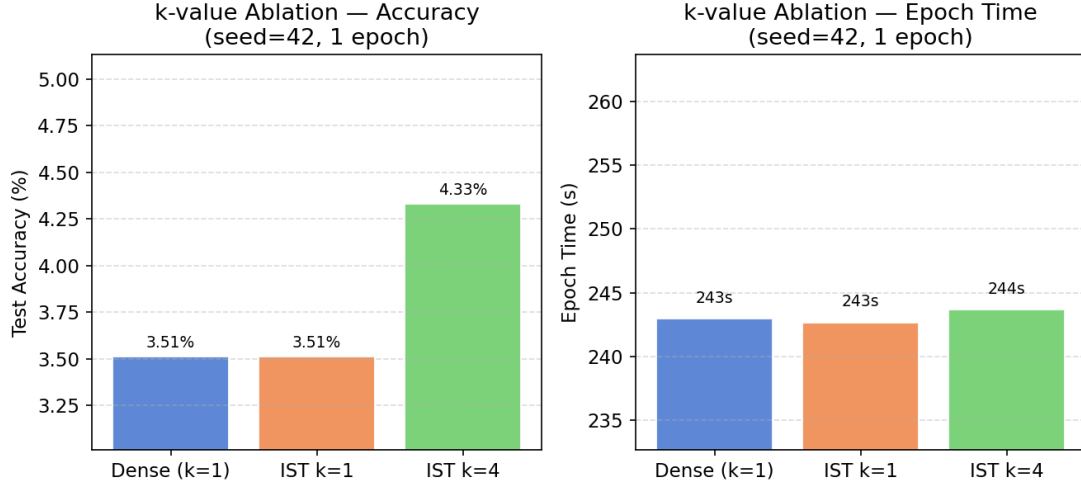


FIGURE 7 – Comparaison entre le chemin dense, IST ($K = 1$) et IST ($K = 4$) pour `seed=42` sur un epoch : précision de validation (gauche) et temps moyen par epoch (droite), sous `Tiny-ImageNet` avec `world_size=2`.

La figure 8 compare ensuite le temps moyen par epoch entre le chemin dense et IST ($K = 4$) sur trois graines $\{42, 43, 44\}$, avec 2 epochs. Dans cette série, IST ($K = 4$) représente en moyenne environ 98.9% du temps du chemin dense, soit un gain de l'ordre de 1%. Cet écart reste modeste et peut encore dépendre de la charge du cluster ainsi que de l'initialisation aléatoire ; il doit donc être lu comme une validation de faisabilité et non comme une conclusion de performance forte.

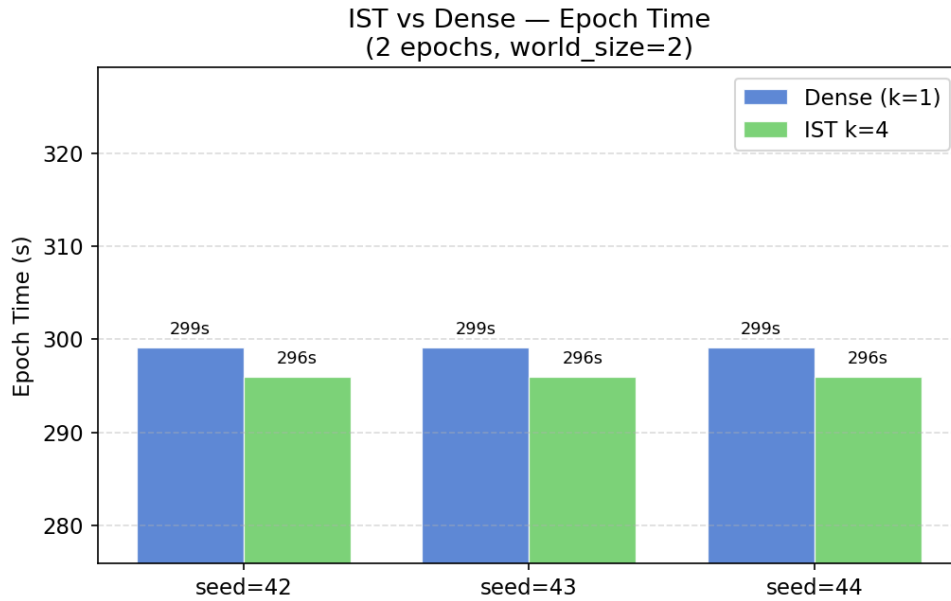


FIGURE 8 – Temps moyen par epoch pour le chemin dense et IST ($K = 4$) sur trois graines aléatoires, avec 2 epochs, sous `Tiny-ImageNet` et `world_size=2`. Dans cette campagne réduite, IST ($K = 4$) est légèrement plus rapide que le chemin dense.

Références

- [1] Yann LeCun, Léon Bottou, Yoshua Bengio, and Patrick Haffner. “Gradient-based learning applied to document recognition.” *Proceedings of the IEEE*, 86(11) :2278–2324, 1998.
- [2] Kazushige Goto and Robert A. van de Geijn. “Anatomy of High-Performance Matrix Multiplication.” *ACM Transactions on Mathematical Software*, 34(3) :12 :1–12 :25, 2008.
- [3] Field G. Van Zee and Robert A. van de Geijn. “BLIS : A Framework for Rapidly Instantiating BLAS Functionality.” *ACM Transactions on Mathematical Software*, 41(3) :14 :1–14 :33, 2015.
- [4] Field G. Van Zee and Tyler M. Smith. “Implementing High-Performance Complex Matrix Multiplication via the 3m and 4m Methods.” *ACM Transactions on Mathematical Software*, 44(1) :7 :1–7 :36, 2017.
- [5] Tze Meng Low, Francisco D. Igual, Tyler M. Smith, and Enrique S. Quintana-Ortí. “Analytical Modeling Is Enough for High-Performance BLIS.” *ACM Transactions on Mathematical Software*, 43(2) :12 :1–12 :18, 2016.
- [6] Tyler M. Smith, Robert A. van de Geijn, Mikhail Smelyanskiy, Jeff R. Hammond, and Field G. Van Zee. “Anatomy of High-Performance Many-Threaded Matrix Multiplication.” *Proceedings of the IEEE 28th International Parallel and Distributed Processing Symposium (IPDPS)*, pages 1049–1059, 2014.
- [7] Ning Qian. “On the Momentum Term in Gradient Descent Learning Algorithms.” *Neural Networks*, 12(1) :145–151, 1999.
- [8] Ilya Loshchilov and Frank Hutter. “Decoupled Weight Decay Regularization.” *International Conference on Learning Representations (ICLR)*, 2019.
- [9] Yangqing Jia, Evan Shelhamer, Jeff Donahue, Sergey Karayev, Jonathan Long, Ross Girshick, Sergio Guadarrama, and Trevor Darrell. “Caffe : Convolutional Architecture for Fast Feature Embedding.” *Proceedings of the 22nd ACM International Conference on Multimedia*, pages 675–678, 2014.
- [10] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. “Deep Residual Learning for Image Recognition.” *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 770–778, 2016.
- [11] Takuya Akiba, Shotaro Sano, Toshihiro Yanase, Takeru Ohta, and Masanori Koyama. “Optuna : A Next-generation Hyperparameter Optimization Framework.” *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 2623–2631, 2019.
- [12] Jia Deng, Wei Dong, Richard Socher, Li-Jia Li, Kai Li, and Li Fei-Fei. “ImageNet : A Large-Scale Hierarchical Image Database.” *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 248–255, 2009.
- [13] Alexander Sergeev and Mike Del Balso. “Horovod : Fast and Easy Distributed Deep Learning in TensorFlow.” *arXiv preprint arXiv :1802.05799*, 2018.
- [14] Dan Alistarh, Demjan Grubic, Jerry Li, Ryota Tomioka, and Milan Vojnovic. “QSGD : Communication-Efficient SGD via Gradient Quantization and Encoding.” *Advances in Neural Information Processing Systems (NeurIPS)*, 30, 2017.

- [15] Binhang Yuan, Cameron R. Wolfe, Chen Dun, Yuxin Tang, Anastasios Kyrillidis, and Chris Jermaine. “Distributed Learning of Fully Connected Neural Networks using Independent Subnet Training.” *Proceedings of the VLDB Endowment*, 15(8) :1581–1590, 2022. <https://www.vldb.org/pvldb/vol15/p1581-wolfe.pdf>.

Glossaire

A Environnement expérimental GEMM, workloads et protocole de recherche

A.1 Environnement expérimental

A.1.1 Plateforme matérielle

Composant	Spécification
Processeur	AMD Ryzen 9 8940HX, architecture x86_64
Cœurs physiques / threads logiques	16 cœurs physiques, 32 threads logiques (2 threads par cœur, SMT)
Restriction expérimentale	Limitation à 8 CPU via <code>taskset -c 0-7</code> , avec vérification du mapping sur les cœurs physiques par <code>lscpu -e=CPU,CORE,SOCKET,NODE</code>
Hiérarchie de cache	L1d 32 KiB/cœur, L2 1 MiB/cœur, L3 64 MiB (partagé)
Fréquence d'exécution	Verrouillée à 4.0 GHz (Turbo Boost désactivé)
Mémoire	30 GiB

TABLE 4 – Configuration matérielle des expériences GEMM

A.1.2 Environnement logiciel

Composant	Version / détails
Système d'exploitation	Fedora Linux 42
Compilateur	GCC 15.2.1, options de compilation <code>-O3 -march=native</code>
Bibliothèque BLAS	OpenBLAS (version système)
Modèle parallèle	OpenMP 5.0
Jeu d'instructions vectorielles	AVX2 / AVX-512 (activé automatiquement via <code>-march=native</code>)

TABLE 5 – Configuration logicielle des expériences GEMM

A.1.3 Protocole de mesure

Le protocole de mesure est le suivant :

- **Préchauffage (warm-up)** : 5 itérations de préchauffage sont exécutées avant chaque série afin de stabiliser l'état des caches d'instructions et de données ;
- **Mesure de temps** : le programme exécute 2000 itérations en boucle interne, chronométrées avec `std::chrono::high_resolution_clock` ;
- **Compteurs matériels** : `perf stat -r 3` répète la mesure 3 fois au niveau externe et collecte le nombre d'instructions, de cycles, de commutations de contexte, de migrations CPU et de

défauts de cache ;

- **Isolation de l’environnement** : les séries critiques sont séparées par un redémarrage du système, un vidage du page cache (`drop_caches`) et un verrouillage de la fréquence CPU via `cpupower`, afin de réduire l’hétérogénéité de l’état système. Le préchauffage reste le principal mécanisme de stabilisation du chemin d’exécution ; le redémarrage et le vidage du page cache servent uniquement à limiter les interférences entre séries.

Le modificateur `:u` (mode utilisateur) de `perf` exclut les commutations de contexte en mode noyau. Le protocole retient donc systématiquement `context-switches` sans modificateur afin de capturer l’ensemble des événements d’ordonnancement. Par ailleurs, les événements AMD uncore L3 sont collectés en mode system-wide et restent sensibles aux processus d’arrière-plan sur des fenêtres courtes ; ils ne sont donc pas retenus pour la comparaison quantitative. Le rapport utilise à la place `cache-misses` et `cache-references` en mode per-task comme indicateurs agrégés de pression mémoire. `cache-misses` sert uniquement à la comparaison relative entre configurations, sans attribution précise à L1, L2 ou L3.

Marge CPU et isolation expérimentale. Les expériences GEMM sont limitées à un budget contrôlé de cœurs CPU, sans occupation complète de tous les threads logiques, afin de laisser une marge au système d’exploitation, au runtime OpenMP, à la collecte `perf` et aux services d’arrière-plan, et de réduire les migrations de threads, les interruptions et la contention non contrôlée. Deux familles de configuration sont retenues : les configurations recommandées, limitées à un budget validé de cœurs physiques, et une configuration de stress à 16 threads avec SMT / hyper-threading.

A.2 Workloads utilisés pour le réglage mono-thread

Quatre familles de workloads représentatives sont définies à partir des formes de couches réellement utilisées dans le MLP et le CNN du projet :

Famille de workload	Couche source	M	K	N
<code>fc_forward_mainstream</code>	MLP FC1 (hidden=128)	32	784	128
<code>fc_head_small_n</code>	MLP FC2 (output)	32	128	10
<code>conv_dx_skinny_k</code>	CNN Conv2 backward	3200	16	150
<code>fc_wide_output</code>	MLP FC1 (hidden=256)	32	784	256

TABLE 6 – Workloads représentatifs utilisés pour le réglage mono-thread (batch=32)

A.3 Protocole de recherche en deux tours

Premier tour (filtrage global des candidats) : pour les 6 noyaux AVX2, plusieurs combinaisons (M_c, N_c, K_c) sont balayées sur chaque workload, soit 272 candidats au total. Chaque candidat est exécuté 1000 fois et évalué à partir du temps médian, avec en complément 5 échantillons `perf stat` pour les compteurs matériels. Le balayage suit une stratégie par étapes : K_c est d’abord exploré à M_c, N_c fixés, puis M_c avec le meilleur K_c , puis N_c dans une grille plus fine, de sorte que les variations de performance restent attribuables à un seul axe paramétrique.

Second tour (raffinement local) : les 5 combinaisons `KernelShape/Kc` les plus performantes sur les principales formes FC au premier tour sont conservées : `8x8/Kc=288`, `8x8/Kc=384`, `4x16/Kc=192`, `4x16/Kc=256`, `4x16/Kc=320`, et la grille de recherche en M_c, N_c est densifiée autour de leur meilleure région (environ 75 combinaisons au total) afin de localiser plus précisément les paramètres optimaux.

B Diagnostic des coûts d’ordonnancement et de synchronisation en configuration 16 threads

Ce diagnostic sert uniquement à fixer le protocole expérimental et ne fait pas partie des conclusions principales sur la performance.

L’élément central de ce contrôle provient d’une expérience d’affinité CPU (OpenBLAS, 16 threads, matrice 128×128 , 5000 itérations) : en ordonnancement libre, on mesure 2,013 commutations de contexte et 13.45 B instructions ; avec `OMP_PROC_BIND=true`, `OMP_PLACES=cores`, les commutations de contexte tombent à 14 (−99.3%) et le nombre total d’instructions à 4.05 B (−69.9%). Cette observation concerne les surcoûts de la configuration 16 threads, dominés ici par l’ordonnancement et l’attente de synchronisation plutôt que par du calcul supplémentaire.

Indicateur	Libre	Cœurs liés	Variation
context switch (CS)	2,013	14	−99.3%
Instructions totales	13.45 B	4.05 B	−69.9%
Écart-type cache misses	1.32%	0.06%	stabilité multipliée par ~ 22

TABLE 7 – Effet de l’affinité CPU sur la performance à 16 threads (matrice 128×128 , 5000 itérations)

Les expériences suivantes sont donc limitées à 1 à 8 threads physiques avec liaison explicite aux cœurs, afin d’éviter que la variabilité d’ordonnancement propre au SMT n’interfère avec la comparaison des backends.

La figure 9 regroupe la comparaison de passage à l’échelle après contrainte d’affinité.

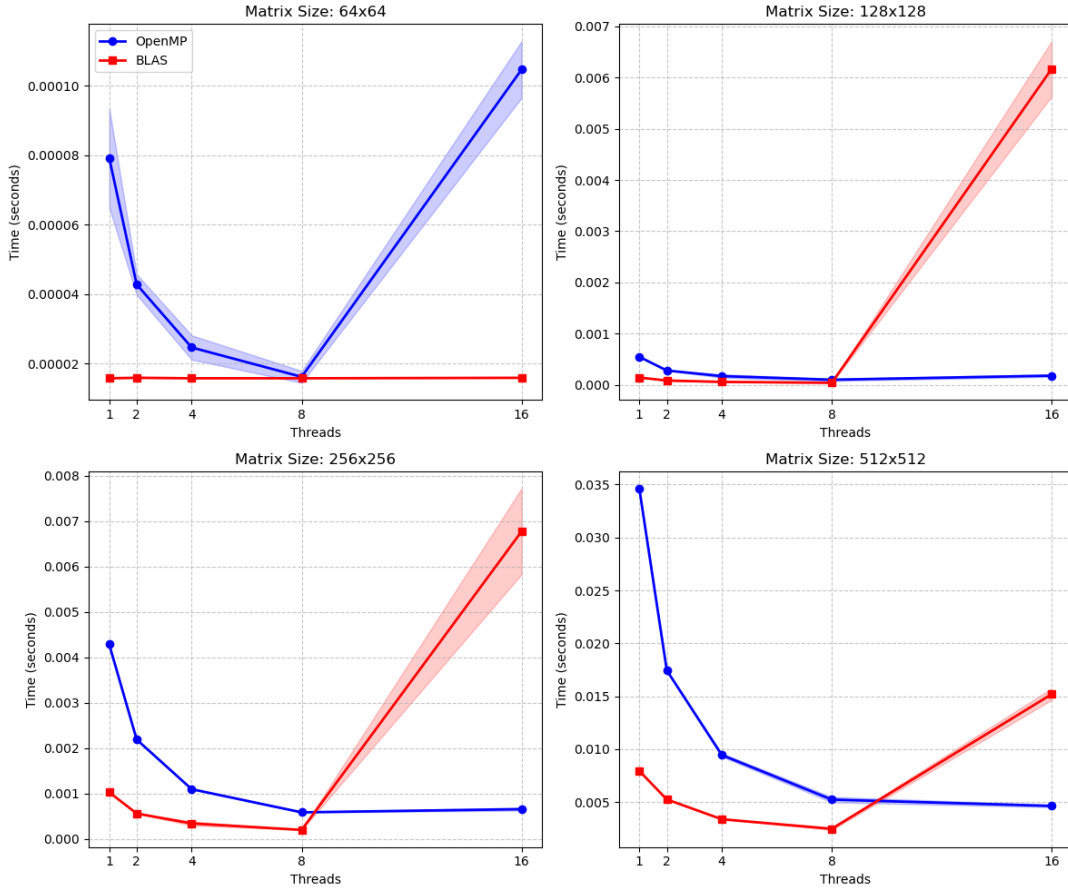


FIGURE 9 – Comparaison de la scalabilité en nombre de threads après contrainte d’affinité CPU (BLAS2). Sur les matrices carrées de 64×64 à 512×512 couvertes par la figure, et avec 8 threads physiques, `blas` fournit le temps mesuré le plus faible parmi les implémentations comparées.

C Le filtrage des paramètres OpenMP blocked / packing

Cette annexe complète le protocole intégral des expériences de la famille blocked présentées après la section 2.2. Les essais portent sur trois variantes : `omp_blocked`, `omp_blocked_packb` et `omp_blocked_packab`. Le filtrage se déroule en deux tours : le premier effectue une sélection de paramètres sur les petites matrices carrées 64, 128, 256 et 512, avec 1, 2, 4 et 8 threads physiques et 1000 répétitions par défaut ; le second réévalue sur 1024 et 2048 le petit nombre de candidats conservés afin de vérifier leur stabilité sur grandes matrices et d’en déduire les paramètres représentatifs.

L’espace de candidats du premier tour n’est pas issu d’une recherche libre complète ; il est construit à partir des contraintes théoriques de résidence en cache données dans le corps du texte. `omp_blocked` teste `BlockSize` $\in \{16, 32, 48, 64, 128, 256\}$, avec les 3 premières valeurs orientées L1 et les 3 suivantes orientées L2 ; `omp_blocked_packb` se limite à `BlockSize` $\in \{16, 32, 48\}$, car l’objectif est d’évaluer une stratégie packed-*B* orientée L1 ; `omp_blocked_packab` n’utilise plus une taille de bloc unique, mais retient parmi `PackM`, `PackN` $\in \{32, 48, 64, 96, 128\}$ et `PackK` $\in \{16, 32, 48, 64, 96\}$ les combinaisons compatibles avec la contrainte de footprint L1, puis applique le filtrage du premier tour sur cet ensemble.

À l’issue du premier tour, le programme agrège d’abord les résultats, puis sélectionne automatiquement les quelques candidats du second tour à partir du temps moyen inter-threads sur

une taille de référence, par défaut la plus grande taille filtrée, soit 512. La figure 10 présente uniquement la synthèse moyenne par thread du premier tour pour `omp_blocked` sur les quatre tailles carrées; elle sert à situer les block sizes les uns par rapport aux autres, sans désigner directement un optimum unique. Les synthèses de candidats et les mesures de revalidation du second tour sont conservées pour vérification ultérieure.

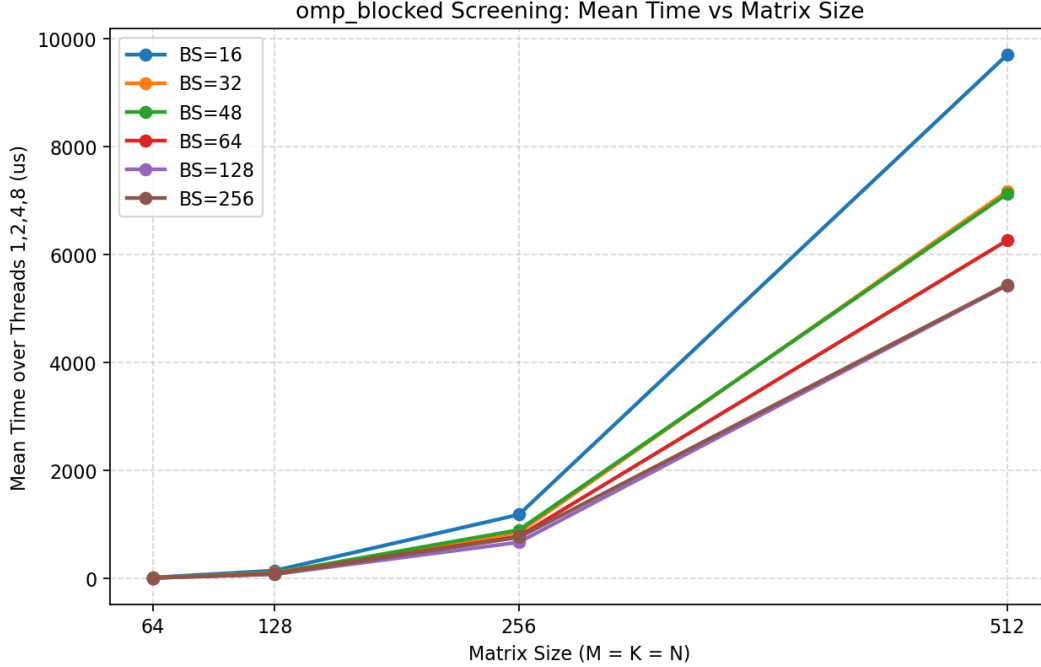


FIGURE 10 – Synthèse du filtrage sur petites matrices pour `omp_blocked` entre 64 et 512. Après moyenne des résultats à 1, 2, 4 et 8 threads physiques, `BlockSize=64` apparaît plus souvent dans la zone favorable sur les matrices les plus grandes, sans optimum unique commun à toutes les tailles de matrices et à tous les nombres de threads.

Le CSV de synthèse `omp_blocked_packb` associé à la figure 10 indique que, dans l'intervalle conservé `BlockSize` $\in \{16, 32, 48\}$, les valeurs 32 et 48 dominent de façon assez stable 16; sur 64×64 , l'avantage revient le plus souvent à 32, sur 128×128 à 48, et sur 256×256 comme sur 512×512 , la tête alterne fréquemment entre 32 et 48. Pour `omp_blocked_packab`, les candidats du second tour se concentrent après filtrage sur un petit nombre de combinaisons orientées L1, et les paramètres représentatifs finalement retenus sont $(\text{PackM}, \text{PackN}, \text{PackK}) = (32, 128, 16)$.

D Synthèse des résultats de réglage AVX2 en mono-thread

La figure 11 illustre un workload représentatif de couche entièrement connectée, `fc_forward_mainstream_n` / `mlp_fc1_b32` ($32 \times 784 \times 128$). Pour chaque forme de micro-kernel, seule la meilleure combinaison de paramètres de blocking associée est conservée.

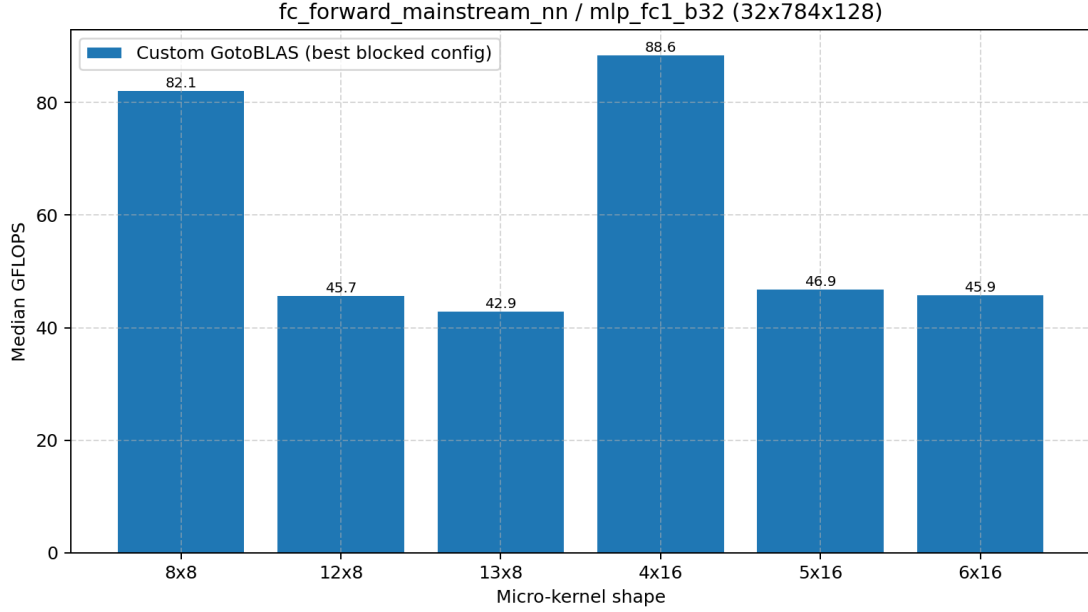


FIGURE 11 – Comparaison mono-thread des GFLOPS pour différentes formes de micro-kernel AVX2 sur le workload de couche entièrement connectée `mlp_fc1_b32`. L'axe horizontal représente la forme du micro-kernel et l'axe vertical la médiane des GFLOPS obtenue avec les meilleurs paramètres de blocking pour cette forme.

Sur ce workload, les formes `4x16` et `8x8` dominent nettement : `4x16` atteint environ 88.6 GFLOPS et `8x8` environ 82.1 GFLOPS, tandis que les autres formes restent pour la plupart entre 43 et 47 GFLOPS.

Famille de workload	Meilleur noyau	M_c	N_c	K_c	Personnalisé (μs)	OpenBLAS (μs)	Écart
fc_forward (b32)	4x16	128	576	320	72.5	60.2	-17%
fc_forward (b64)	4x16	192	512	256	132.6	110.1	-17%
fc_head (b32)	4x16	—	—	—	4.76	1.14	-76%
conv_dx (b32)	8x8	160	640	288	744.3	647.3	-13%
fc_wide (b32)	4x16	128	768	256	136.5	122.2	-10%

TABLE 8 – Réglage AVX2 en mono-thread : meilleure configuration par famille de workloads et comparaison à OpenBLAS en mono-thread

Sur l'ensemble des workloads résumés au tableau 8, l'écart à OpenBLAS reste généralement compris entre 10 et 24% sur les formes courantes, mais atteint 66 à 78% sur les très petits N (comme `fc_head`), où les traitements de bord dominent.

E Dérivation de l'ensemble de candidats AVX2 pour les blocked sizes

Cette annexe complète la justification de la réduction de l'espace de recherche (M_r, N_r, K_c, M_c, N_c) présentée dans le corps du rapport.

E.1 Dérivation de m_r et n_r liée à l'implémentation

Le micro-kernel courant exécute à chaque itération p $C_r(i, :) \leftarrow C_r(i, :) + A_r(i, p) B_r(p, :)$, où $0 \leq i < m_r$. Ici, $B_r(p, :)$ est chargé d'un seul coup dans un registre vectoriel AVX2, tandis que

$A_r(i, p)$ participe au FMA par diffusion scalaire. Le bloc sémantique $m_r \times n_r$ reste inchangé ; les lanes SIMD correspondent à la dimension n_r , ce qui donne une orientation des registres différente de celle de certains exemples de la littérature.

On note N_{vec} le nombre de scalaires contenus dans un registre vectoriel. Pour AVX2 en `float32`, on a $N_{\text{vec}} = \frac{256}{32} = 8$. Dans le cadre de l'optimisation paramétrique, n_r doit au minimum respecter la granularité de vectorisation et se prend généralement comme multiple entier de N_{vec} , soit $n_r = t N_{\text{vec}}$ avec $t \in \mathbb{Z}_{\geq 1}$. Si un même $B_r(p, :)$ est porté par deux registres YMM au cours de la même itération, n_r peut aussi valoir 16. La suite ne considère que $t \in \{1, 2\}$, donc $n_r \in \{8, 16\}$, sans traiter pour l'instant des micro-kernels plus larges en direction n , tels que $n_r = 24, 32$.

Les contraintes sur m_r proviennent du masquage de latence et du budget de registres. La condition de base issue des travaux BLIS sur les micro-kernels peut s'écrire $m_r n_r \geq N_{\text{vec}} L_{\text{vfma}} N_{\text{vfma}} = 8 \times 4 \times 2 = 64$, où L_{vfma} est la latence dépendante du FMA vectoriel, ici fixée à 4 pour un modèle de premier ordre, et N_{vfma} le nombre de FMA vectoriels émis par cycle, ici fixé à 2. Cette relation fournit la borne inférieure côté débit.

Le budget de registres ne peut plus s'écrire sous une forme unique indépendante de n_r , car lorsque $n_r = 8$, chaque ligne d'accumulateurs n'occupe qu'un YMM, alors que pour $n_r = 16$, chaque ligne en occupe 2. En posant $t = n_r / N_{\text{vec}} = n_r / 8$, le nombre de registres accumulateurs vaut $m_r t$. Le budget de registres doit donc au minimum dépendre de t : si une itération conserve seulement 1 registre temporaire vectoriel pour B , on obtient $m_r t + 1 + \delta \leq N_{\text{reg}} = 16$; et, avec un modèle plus conservateur pour $n_r = 16$, en considérant que les deux sous-vecteurs de B doivent être conservés simultanément, on obtient $m_r t + t + \delta \leq N_{\text{reg}} = 16$. où N_{reg} est le nombre de registres vectoriels disponibles, et δ la marge réservée au broadcast, à la génération d'adresses et à l'ordonnancement, avec $\delta \in [1, 3]$.

Il faut donc distinguer les cas. Quand $n_r = 8$, soit $t = 1$, la borne de débit donne $m_r \cdot 8 \geq 64 \Rightarrow m_r \geq 8$, tandis que le budget de registres optimiste donne $m_r + 1 + \delta \leq 16$. Pour $\delta \in [1, 3]$, on obtient $8 \leq m_r \leq 13$, d'où l'ensemble conservé dans ce modèle : $m_r \in \{8, 9, 10, 11, 12, 13\}$. Quand $n_r = 16$, soit $t = 2$, la borne de débit donne $m_r \cdot 16 \geq 64 \Rightarrow m_r \geq 4$. Le budget de registres optimiste devient $2m_r + 1 + \delta \leq 16$, et sa version plus conservatrice s'écrit $2m_r + 2 + \delta \leq 16$. Sous cet ensemble de contraintes, l'intervalle faisable de m_r est plus étroit pour $n_r = 16$; en retenant les candidats les plus sûrs, on obtient $m_r \in \{4, 5, 6\}$, $m_r = 7$ n'est conservable que dans l'hypothèse plus optimiste où $n_r = 16$ et un seul registre temporaire vectoriel de B est gardé.

Le domaine faisable de cette section se résume alors, selon les cas en $n_r/8$, à :

$$\begin{cases} n_r \in \{8, 16\}, \\ m_r n_r \geq 64, \\ m_r \frac{n_r}{8} + 1 + \delta \leq 16 & \text{(version optimiste ; pour } n_r = 16, \text{ un seul registre temporaire pour } B), \\ m_r \frac{n_r}{8} + \frac{n_r}{8} + \delta \leq 16 & \text{(version conservatrice ; pour } n_r = 16, \text{ tous les sous-vecteurs de } B). \end{cases}$$

On en déduit que les couples candidats (m_r, n_r) du domaine faisable initial s'écrivent

$$\{(8, 8), (9, 8), (10, 8), (11, 8), (12, 8), (13, 8), (4, 16), (5, 16), (6, 16)\},$$

où $(7, 16)$ ne reste possible que sous un budget de registres plus optimiste ; il n'est donc pas inclus dans l'ensemble de candidats par défaut. Pour contenir la taille de la recherche ultérieure, l'ensemble finalement retenu pour (m_r, n_r) est $\{(8, 8), (12, 8), (13, 8), (4, 16), (5, 16), (6, 16)\}$.

E.2 Modèle de cache et génération des candidats pour k_c

B.2.1 Centres candidats de k_c fondés sur L1

Pour un couple (m_r, n_r) donné, k_c fixe la profondeur de packed $A_r \in \mathbb{R}^{m_r \times k_c}$ et de $B_r \in \mathbb{R}^{k_c \times n_r}$. Le rapport construit d'abord des centres candidats à partir de l'occupation des panneaux côté L1, puis en déduit un ensemble discret à l'aide des contraintes d'alignement et de page footprint.

Pour AVX2 en `float32`, en prenant la cache line comme unité et 1024B par voie de L1, la pression approximative sur les sets des panneaux A_r et B_r s'écrit proportionnellement à $m_r k_c$ et $n_r k_c$. Après simplification, la borne supérieure du budget de sets disponible pour A_r est $C_{A_r}^{\max} = \left\lfloor \frac{7}{1+n_r/m_r} \right\rfloor$, ce qui donne le centre candidat $k_c^{\text{center}} \approx \frac{1024 C_{A_r}^{\max}}{m_r}$.

k_c^{center} n'est utilisé ici que comme centre candidat sur le matériel courant. Le micro-kernel lit d'abord $B_r(p, :)$ puis diffuse les scalaires de $A_r(:, p)$ à chaque itération p ; le modèle L1 sert donc, dans ce rapport, à fournir une valeur centrale et le point de départ de la discrétisation.

B.2.2 Contraintes sur k_c fondées sur le TLB / page footprint

La relation L1 / set-mapping ne suffit pas à elle seule pour fixer k_c , car l'augmentation de k_c accroît simultanément le page footprint des micro-panneaux et des macro-panneaux. Si la taille de page vaut P , le footprint au niveau micro-panneau vérifie approximativement

$$\frac{m_r k_c S_{\text{data}}}{P} \lesssim E_{A_r, \text{TLB}}, \quad \frac{k_c n_r S_{\text{data}}}{P} \lesssim E_{B_r, \text{TLB}},$$

Cette forme de filtrage reste indicative dans le rapport, qui passe ensuite à un filtrage au niveau macro-panneau. Pour rendre la couche TLB exploitable sous forme de règle, on utilise ici $m_c = 128$ et $n_c = 512$ comme volumes de panneaux provisoires; ils servent uniquement au TLB-aware pruning de ce tour. Pour $P = 4096$, on obtient

$$\begin{aligned} \text{pages}(A_c) &\approx \left\lceil \frac{m_c k_c S_{\text{data}}}{P} \right\rceil = \left\lceil \frac{128 \cdot k_c \cdot 4}{4096} \right\rceil = \left\lceil \frac{k_c}{8} \right\rceil, \\ \text{pages}(B_c) &\approx \left\lceil \frac{k_c n_c S_{\text{data}}}{P} \right\rceil = \left\lceil \frac{k_c \cdot 512 \cdot 4}{4096} \right\rceil = \left\lceil \frac{k_c}{2} \right\rceil. \end{aligned}$$

Sur la machine étudiée, les L1 DTLB et L2 DTLB disposent respectivement de 72 et 3072 entrées. Comme le nombre de pages de B_c reste très inférieur à la capacité du L2 DTLB pour les candidats de cette section, le filtrage réellement discriminant provient de A_c , privé à chaque thread. Afin de laisser une marge pour la pile, le bloc C , le code et les autres données, on retient le budget effectif conservateur $E_{A, \text{eff}} = 0.75 \times 72 = 54$, avec la règle de pruning suivante : si un candidat vérifie $\text{pages}(A_c) > 54$, il est retiré de l'ensemble expérimental du premier tour; si $\text{pages}(A_c) \leq 54$ et $\text{pages}(B_c) \ll 3072$, il passe le contrôle de cette couche TLB. Cette couche n'assure qu'un filtrage approximatif et ne génère pas de nouveau centre.

B.2.3 Génération des candidats k_c sous contraintes L1 et TLB

La génération des candidats suit deux étapes. La première produit, à partir du modèle L1, un ensemble discret initial autour du centre candidat. En notant g_k la granularité d'alignement de l'implémentation pour k_c , on obtient

$$\mathcal{K}_{L1}(m_r, n_r) = \text{Align}_{g_k}(\{\alpha k_c^{\text{center}}(m_r, n_r) : \alpha \in \mathcal{A}_k\}),$$

où g_k est déterminé par le pas du packed panel et la granularité des boucles externes en blocs, tandis que $\mathcal{A}_k$ désigne un voisinage multiplicatif fini autour de 1. Pour contenir la taille

du premier tour, on prend ici l'instanciation conservatrice $g_k = 32$ et $\mathcal{A}_k = \{0.75, 1.00, 1.25\}$.

La seconde étape applique un filtrage approximatif à $\mathcal{K}_{L1}(m_r, n_r)$ au moyen des contraintes TLB / page footprint, ce qui donne l'ensemble expérimental du premier tour

$$\mathcal{K}_{\text{pruned}}(m_r, n_r) \subseteq \mathcal{K}_{L1}(m_r, n_r).$$

Ici, k_c^{center} , $\mathcal{K}_{L1}$ et $\mathcal{K}_{\text{pruned}}$ dépendent tous du couple (m_r, n_r) considéré. Les expériences suivantes peuvent soit explorer séparément chaque (m_r, n_r) , soit fusionner les ensembles dans un pool commun, sans changer la hiérarchie : génération par L1, puis pruning par TLB.

Pour les 6 couples (m_r, n_r) finalement retenus à la section B.1, l'application de $C_{A_r}^{\text{max}} = \left\lfloor \frac{7}{1+n_r/m_r} \right\rfloor$ et de $k_c^{\text{center}} \approx \frac{1024 C_{A_r}^{\text{max}}}{m_r}$ conduit au tableau 9. Le tableau présente d'abord $\mathcal{K}_{L1}(m_r, n_r)$, puis $\mathcal{K}_{\text{pruned}}(m_r, n_r)$ après filtrage par la couche TLB ; les candidats trop grands, pour lesquels le nombre de pages du A_c provisoire dépasse 54, sont effectivement supprimés.

(m_r, n_r)	$C_{A_r}^{\text{max}}$	k_c^{center}	$\mathcal{K}_{L1}$	$\mathcal{K}_{\text{pruned}}$
(8, 8)	3	384.0	{288, 384, 480}	{288, 384}
(12, 8)	4	341.3	{256, 352, 416}	{256, 352, 416}
(13, 8)	4	315.1	{224, 320, 384}	{224, 320, 384}
(4, 16)	1	256.0	{192, 256, 320}	{192, 256, 320}
(5, 16)	1	204.8	{160, 192, 256}	{160, 192, 256}
(6, 16)	1	170.7	{128, 160, 224}	{128, 160, 224}

TABLE 9 – Génération des candidats k_c et résultat du pruning TLB

Ces $\mathcal{K}_{\text{pruned}}(m_r, n_r)$ constituent les candidats du premier tour pour la recherche sur k_c . Ils proviennent d'un centre calculé sur le matériel courant, d'un alignement à 32 et d'un voisinage multiplicatif fini, puis d'un filtrage page footprint / TLB ; par exemple, le $k_c = 480$ issu du facteur 1.25 est retiré sur la ligne (8, 8), car le nombre de pages du A_c provisoire atteint 60, au-dessus du budget effectif de 54 entrées L1 DTLB.

Les contraintes de cette section se résument à

$$\begin{cases} C_{A_r} + C_{B_r} \leq W_{L1} - 1, \\ C_{A_r} \leq \left\lfloor \frac{W_{L1} - 1}{1 + n_r/m_r} \right\rfloor, \\ k_c^{\text{center}} \approx \frac{1024 C_{A_r}}{m_r}, \\ \mathcal{K}_{L1}(m_r, n_r) = \text{Align}_{g_k}(\{\alpha k_c^{\text{center}}(m_r, n_r) : \alpha \in \mathcal{A}_k\}), \\ \mathcal{K}_{\text{pruned}}(m_r, n_r) \subseteq \mathcal{K}_{L1}(m_r, n_r). \end{cases}$$

E.3 Génération des candidats de base mono-thread pour m_c/n_c

B.3.1 Contraintes sur m_c/n_c fondées sur le cache

m_c et n_c sont construits à partir des candidats (m_r, n_r, k_c) déjà filtrés : une fois la forme du micro-kernel et la profondeur de panneau le long de k fixées, m_c détermine la hauteur du jeu de travail privé $A_c \in \mathbb{R}^{m_c \times k_c}$ et n_c la largeur du jeu de travail partagé $B_c \in \mathbb{R}^{k_c \times n_c}$. La présente section transforme ces contraintes de cache / TLB en ensembles de candidats de base pour le mono-thread ou le parallélisme faible.

Dans l'implémentation courante, A_c est traité comme jeu de travail privé à chaque thread, tandis que B_c correspond à un panneau partagé en lecture sur le chemin multithread ; la section raisonne donc d'abord sur leurs volumes au titre des candidats de base mono-thread. Leurs

tailles en octets sont

$$|A_c| = m_c k_c S_{\text{data}}, \quad |B_c| = k_c n_c S_{\text{data}}.$$

La hiérarchie de cache conduit alors aux contraintes d'échelle les plus directes

$$m_c k_c S_{\text{data}} \lesssim \rho_A S_{\text{private}}, \quad k_c n_c S_{\text{data}} \lesssim \rho_B S_{\text{shared}}.$$

où $\rho_A, \rho_B \in (0, 1)$ sont des coefficients de modélisation destinés à conserver une marge pour les conflits, le code et les autres jeux de travail ; ces relations donnent les bornes d'échelle de base $m_c^{\max}$ et $n_c^{\max}$. Sur la machine courante, le modèle mono-thread de base prend une capacité L2 privée par cœur $S_{\text{private}} = 1$ MiB ; pour borner le budget d'un panneau B_c , il retient aussi un budget effectif conservateur de cache partagé $\rho_B S_{\text{shared}} = 1$ MiB. Avec $\rho_A = 1/4$, on obtient

$$m_c^{\max}(k_c) \approx \frac{\rho_A S_{\text{private}}}{k_c S_{\text{data}}} = \frac{256 \text{ KiB}}{4k_c} = \frac{65536}{k_c},$$

$$n_c^{\max}(k_c) \approx \frac{\rho_B S_{\text{shared}}}{k_c S_{\text{data}}} = \frac{1 \text{ MiB}}{4k_c} = \frac{262144}{k_c}.$$

La couche cache sert donc ici à fournir les bornes d'échelle de base $m_c^{\max}(k_c)$ et $n_c^{\max}(k_c)$ en fonction de k_c , autour desquelles seront générés les candidats discrets.

B.3.2 Contraintes sur m_c/n_c fondées sur le TLB / page footprint

Les bornes de cache ne suffisent pas à elles seules, car la couverture en pages de A_c et B_c augmente avec m_c et n_c . Si la taille de page vaut P , leur couverture approchée est

$$\frac{m_c k_c S_{\text{data}}}{P}, \quad \frac{k_c n_c S_{\text{data}}}{P},$$

Les conditions approximatives de pruning côté TLB deviennent alors

$$\frac{m_c k_c S_{\text{data}}}{P} \lesssim E_{A,\text{TLB}}, \quad \frac{k_c n_c S_{\text{data}}}{P} \lesssim E_{B,\text{TLB}}.$$

Dans cette section, ces relations jouent le rôle d'une seconde couche de pruning. Le code courant ne contrôle ni huge pages, ni STLB, ni appartenance NUMA, ni affinité de thread ; elles restent donc utilisées comme contraintes approchées. Pour la machine étudiée, on prend $P = 4096$, 72 entrées en L1 DTLB, 3072 en L2 DTLB, et les budgets effectifs conservateurs $E_{A,\text{eff}} = 54$ et $E_{B,\text{eff}} = 192$; la valeur 192 sert à éliminer dès le premier tour les panneaux B_c trop larges. On obtient alors

$$\text{pages}(A_c) \approx \frac{m_c k_c}{1024}, \quad \text{pages}(B_c) \approx \frac{k_c n_c}{1024}.$$

La règle de TLB-aware pruning employée ici est donc la suivante : si un candidat m_c vérifie $\text{pages}(A_c) > 54$, il est retiré de $\mathcal{M}_{\text{cache}}$; si un candidat n_c vérifie $\text{pages}(B_c) > 192$, il est retiré de $\mathcal{N}_{\text{cache}}$. Le choix $E_{B,\text{eff}}$ reste inférieur à la capacité matérielle du L2 DTLB afin d'éliminer dès la recherche de base du premier tour les panneaux B_c trop larges.

B.3.3 Génération des candidats de base mono-thread sous contraintes cache et TLB

Pour un candidat (m_r, n_r, k_c) donné, les bornes de cache génèrent d'abord les ensembles discrets initiaux

$$\mathcal{M}_{\text{cache}}(m_r, n_r, k_c) = \text{Align}_{g_m}^\downarrow(\{\beta m_c^{\max}(k_c) : \beta \in \mathcal{A}_m\}),$$

$$\mathcal{N}_{\text{cache}}(m_r, n_r, k_c) = \text{Align}_{g_n}^\downarrow(\{\gamma n_c^{\max}(k_c) : \gamma \in \mathcal{A}_n\}),$$

puis un pruning TLB / page footprint produit

$$\mathcal{M}_{\text{pruned}} \subseteq \mathcal{M}_{\text{cache}}, \quad \mathcal{N}_{\text{pruned}} \subseteq \mathcal{N}_{\text{cache}}.$$

Pour contenir la taille de la recherche de base du premier tour en mono-thread, on prend $g_m = 32$, $g_n = 64$ et $\mathcal{A}_m = \mathcal{A}_n = \{0.5, 0.75, 1.0\}$. Ici, $\text{Align}^\downarrow$ désigne l'alignement vers le bas sur le multiple de granularité le plus proche qui ne dépasse pas la valeur cible. En appliquant cette même règle à toutes les combinaisons (m_r, n_r, k_c) conservées à la section B.2, on obtient les candidats de base mono-thread du tableau 10.

(m_r, n_r, k_c)	$m_c^{\max}$	$\mathcal{M}_{\text{cache}}$	$\mathcal{M}_{\text{pruned}}$	$n_c^{\max}$	$\mathcal{N}_{\text{cache}}$	$\mathcal{N}_{\text{pruned}}$
(8, 8, 288)	227.6	{96, 160, 224}	{96, 160}	910.2	{448, 640, 896}	{448, 640}
(8, 8, 384)	170.7	{64, 128, 160}	{64, 128}	682.7	{320, 512, 640}	{320, 512}
(12, 8, 256)	256.0	{128, 192, 256}	{128, 192}	1024.0	{512, 768, 1024}	{512, 768}
(12, 8, 352)	186.2	{64, 128, 160}	{64, 128}	744.7	{320, 512, 704}	{320, 512}
(12, 8, 416)	157.5	{64, 96, 128}	{64, 96, 128}	630.2	{256, 448, 576}	{256, 448}
(13, 8, 224)	292.6	{128, 192, 288}	{128, 192}	1170.3	{576, 832, 1152}	{576, 832}
(13, 8, 320)	204.8	{96, 128, 192}	{96, 128}	819.2	{384, 576, 768}	{384, 576}
(13, 8, 384)	170.7	{64, 128, 160}	{64, 128}	682.7	{320, 512, 640}	{320, 512}
(4, 16, 192)	341.3	{160, 256, 320}	{160, 256}	1365.3	{640, 1024, 1344}	{640, 1024}
(4, 16, 256)	256.0	{128, 192, 256}	{128, 192}	1024.0	{512, 768, 1024}	{512, 768}
(4, 16, 320)	204.8	{96, 128, 192}	{96, 128}	819.2	{384, 576, 768}	{384, 576}
(5, 16, 160)	409.6	{192, 288, 384}	{192, 288}	1638.4	{768, 1216, 1600}	{768, 1216}
(5, 16, 192)	341.3	{160, 256, 320}	{160, 256}	1365.3	{640, 1024, 1344}	{640, 1024}
(5, 16, 256)	256.0	{128, 192, 256}	{128, 192}	1024.0	{512, 768, 1024}	{512, 768}
(6, 16, 128)	512.0	{256, 384, 512}	{256, 384}	2048.0	{1024, 1536, 2048}	{1024, 1536}
(6, 16, 160)	409.6	{192, 288, 384}	{192, 288}	1638.4	{768, 1216, 1600}	{768, 1216}
(6, 16, 224)	292.6	{128, 192, 288}	{128, 192}	1170.3	{576, 832, 1152}	{576, 832}

TABLE 10 – Résultat du pruning sur M_c/N_c pour les candidats de base mono-thread du premier tour

Les ensembles $\mathcal{M}_{\text{pruned}}$ et $\mathcal{N}_{\text{pruned}}$ du tableau 10 constituent la base de candidats pour le mono-thread ou le parallélisme faible. Ils suivent une structure à deux niveaux : le cache fournit l'échelle initiale, puis le TLB retire les points trop larges. Les expériences suivantes examinent ensuite comment resserrer encore ces candidats.

Le coarse screening mono-thread du premier tour couvre l'ensemble des combinaisons (m_r, n_r, k_c) conservées ; sur cette base, seules les 5 combinaisons **KernelShape/Kc** localement gagnantes sont prolongées au second tour. La grille plus dense utilisée pour M_c/N_c au second tour est donnée au tableau 11.

Noyau	m_r	n_r	k_c	Candidats M_c	Candidats N_c	Nb. de combinaisons
8x8	8	8	288	{96, 128, 160, 192}	{448, 512, 576, 640}	16
8x8	8	8	384	{64, 96, 128}	{320, 384, 448, 512}	12
4x16	4	16	192	{160, 192, 224, 256, 288}	{640, 768, 896, 1024}	20
4x16	4	16	256	{128, 160, 192}	{512, 576, 640, 704, 768}	15
4x16	4	16	320	{96, 128, 160}	{384, 448, 512, 576}	12

 TABLE 11 – Grille de candidats M_c/N_c utilisée pour le filtrage fin du second tour

Le nombre total de candidats au second tour est donc $16 + 12 + 20 + 15 + 12 = 75$. Ces combinaisons ajoutent les points intermédiaires autour des régions déjà identifiées comme favorables au premier tour, ce qui renforce la stabilité des conclusions sur les blocked sizes en mono-thread.

F Complément sur la heatmap de réglage thread-aware $kCacheBlockM/N_c$

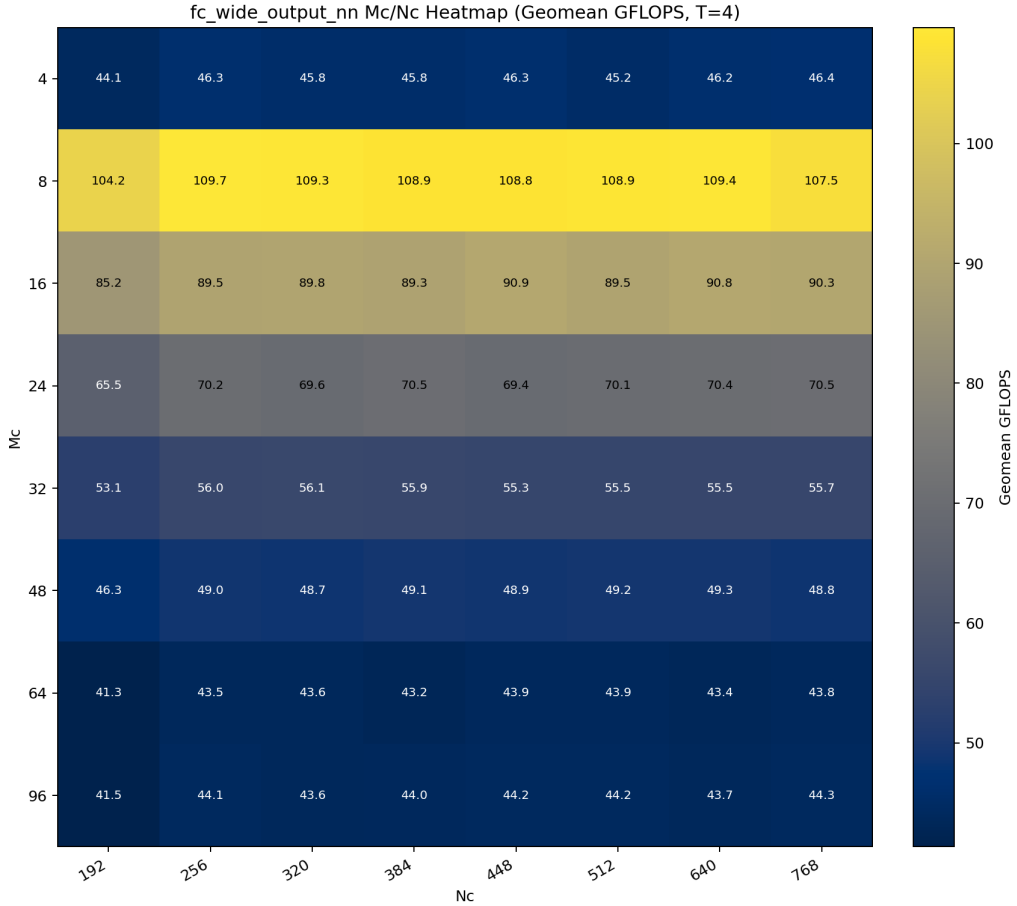


FIGURE 12 – Heatmap représentative du réglage thread-aware de $kCacheBlockM/N_c$ ($T = 4$, $fc_wide_output_nn$). La zone de haute performance se concentre surtout sur la ligne $kCacheBlockM = 8$ (soit $M_c = 64$ effectif), tandis que la direction N_c forme un plateau relativement plat.

G Expérience complémentaire de montée en charge sur matrices carrées fixes

En complément des workloads du projet décrits plus haut, une expérience supplémentaire de montée en charge a été menée sur quatre tailles fixes de matrices carrées : 64×64 , 128×128 , 256×256 et 512×512 . La figure 13 rassemble les résultats de `blas`, `omp`, `omp_blocked`, `omp_blocked_packab`, `omp_gotoblas_avx2` et du chemin AVX-512 unroll+prefetch sur l'intervalle 1–8 threads physiques.

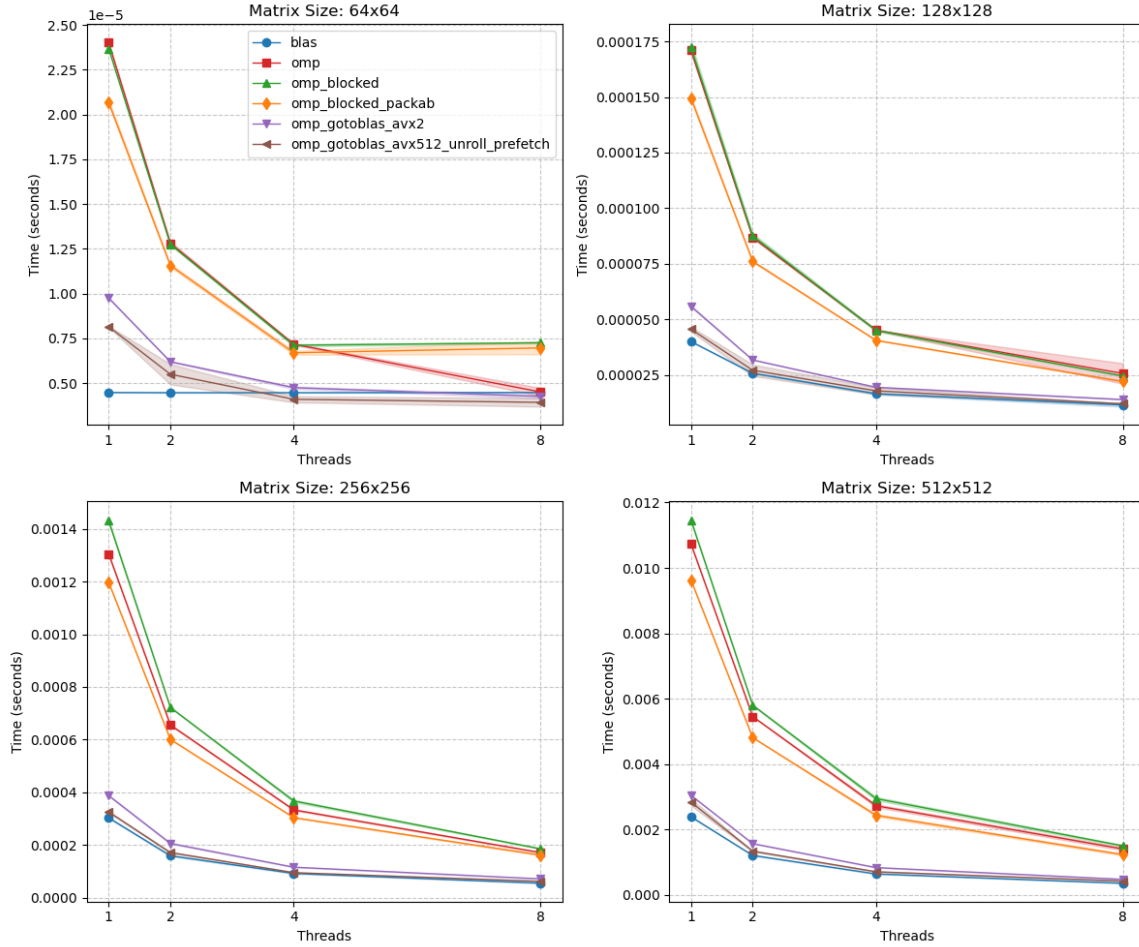


FIGURE 13 – Expérience complémentaire de montée en charge sur matrices carrées fixes avec plusieurs implémentations. Les quatre sous-figures correspondent respectivement à 64×64 , 128×128 , 256×256 et 512×512 ; l'axe vertical représente le temps d'exécution et l'axe horizontal le nombre de threads.

La figure 13 montre que, pour 64×64 , `blas` reste proche d'un comportement quasi mono-thread et varie peu avec le nombre de threads, tandis que les chemins OpenMP n'atteignent généralement ce niveau qu'autour de 4 à 8 threads. Entre 128×128 et 512×512 , les courbes de temps de `omp_gotoblas_avx2` et de `omp_gotoblas_avx512_unroll_prefetch` décroissent plus rapidement; parmi les tailles fixes et l'intervalle 1–8 threads représentés ici, la seconde variante reste en général légèrement meilleure que le chemin AVX2.